

UNIVERSIDAD DE ALMERIA

ESCUELA SUPERIOR DE INGENIERÍA

autoAI:recomendador
de vehículos mediante
LLM y web scraping

Curso: 2025/2026

Alumno/a:

Adrián Martínez Granados

Director/es:

Jesús Manuel Almendros Jiménez

ÍNDICE GENERAL

	Página
Resumen y Abstract	xii
1. Introducción	1
1.1. Contexto y motivación	1
1.2. Descripción del problema	1
1.3. Objetivos del trabajo	1
1.3.1. Objetivo general	1
1.3.2. Objetivos específicos	1
1.4. Estructura de la memoria	1
2. Estado del arte	3
2.1. Buscadores y agregadores de coches existentes	3
2.2. Modelos de lenguaje y procesamiento de lenguaje natural	3
2.3. Web scraping para la obtención de datos	3
2.4. Tecnologías web modernas para aplicaciones interactivas	3
2.5. Conclusiones del análisis	3
3. Análisis de requisitos	5
3.1. Descripción general del sistema	5
3.2. Actores del sistema	6
3.3. Requisitos funcionales	7
3.3.1. Catálogo y consulta estructurada	7
3.3.2. Asistente conversacional	7
3.3.3. Datos del usuario en el navegador	8
3.3.4. Extracción, normalización e ingesta de datos	8
3.4. Requisitos no funcionales	10
3.4.1. Usabilidad y accesibilidad	10
3.4.2. Fiabilidad y robustez	11
3.4.3. Seguridad	11
3.4.4. Rendimiento	12

3.4.5. Mantenibilidad y despliegue	12
3.5. Casos de uso	12
3.5.1. CU-01: Explorar el catálogo de vehículos	12
3.5.2. CU-02: Consultar y comparar fichas de vehículos	13
3.5.3. CU-03: Consultar al asistente conversacional	13
3.5.4. CU-04: Gestionar la lista de favoritos	14
3.5.5. CU-05: Gestionar el historial de conversaciones	14
3.5.6. CU-06: Ajustar las preferencias de accesibilidad	14
3.5.7. CU-07: Comparar opciones mediante visualizaciones	15
4. Metodología y planificación	17
4.1. Metodología de desarrollo	17
4.1.1. Principios y prácticas adoptadas	17
4.1.2. Gestión del cambio: desviaciones respecto al plan inicial	18
4.2. Planificación temporal y fases del proyecto	20
4.2.1. Plan previsto frente a ejecución real	21
4.3. Herramientas y entorno de trabajo	23
4.4. Control de versiones e integración continua	24
4.4.1. Estrategia de control de versiones	24
4.4.2. Integración continua con GitHub Actions	24
5. Diseño del sistema	27
5.1. Arquitectura general	27
5.2. Modelo de datos	29
5.2.1. Diseño de la base de datos	30
5.2.2. Normalización de los datos de los vehículos	32
5.3. Diseño del <i>backend</i> (API REST)	36
5.4. Diseño del pipeline de consulta con IA	37
5.5. Diseño de la interfaz de usuario	42
6. Implementación	49
6.1. Obtención de datos: scraping de km77.com	49
6.1.1. Diseño del <i>spider</i> con Scrapy	49
6.1.2. Control de carga y respeto al portal de origen	51
6.1.3. Automatización periódica del scraping	51
6.2. Proceso ETL y carga en la base de datos	52
6.2.1. Validación y modelos de datos	52
6.2.2. Saneamiento y unificación de vocabularios	52
6.2.3. Persistencia idempotente y tolerante a fallos	54
6.2.4. Esquema de la base de datos	54

6.3. Backend con FastAPI	54
6.3.1. Endpoints del catálogo y comparador	55
6.3.2. El repositorio y la construcción segura de consultas	55
6.3.3. Endpoints de servicio y configuración	55
6.4. Integración de la inteligencia artificial	56
6.4.1. Traducción de lenguaje natural a filtros estructurados	56
6.4.2. Bucle de <i>tool-calling</i> sobre datos reales	57
6.4.3. Generación de respuestas explicadas	58
6.5. Frontend con React	58
6.5.1. Interfaz conversacional	59
6.5.2. Catálogo, filtros y comparador	59
6.5.3. Favoritos e historial de consultas	59
6.5.4. Preferencias de accesibilidad	60
6.6. Contenerización y despliegue con Docker	60
7. Pruebas y evaluación	63
7.1. Estrategia de pruebas	63
7.1.1. Niveles y tipos de prueba	63
7.1.2. Herramientas e infraestructura de pruebas	64
7.1.3. Pruebas como parte de la integración continua	65
7.2. Pruebas del backend y del pipeline de IA	65
7.2.1. Pruebas de la normalización	66
7.2.2. Pruebas de integración del repositorio	67
7.2.3. Pruebas del pipeline de IA	67
7.2.4. Pruebas de propiedad de la búsqueda exploratoria	68
7.2.5. Pruebas de la ingesta tolerante a fallos y de la seguridad	69
7.3. Pruebas del <i>frontend</i>	69
7.3.1. Formato de datos y persistencia local	69
7.3.2. Componentes de interfaz	70
7.3.3. Cobertura de código	70
7.4. Evaluación de la calidad de los resultados	72
7.4.1. La invariante de veracidad como criterio central	72
7.4.2. Evaluación cualitativa mediante consultas representativas	72
7.4.3. Cobertura de los requisitos por las pruebas	74
7.5. Discusión de los resultados	74
8. Conclusiones y trabajo futuro	77
8.1. Cumplimiento de los objetivos	77
8.2. Conclusiones	77

8.3. Líneas de trabajo futuro	77
BIBLIOGRAFÍA	77

A. Contenido de una entrada en .bib	81
B. Tipos de referencias	83

ÍNDICE DE FIGURAS

3.1. Diagrama de casos de uso del usuario final.	15
4.1. Diagrama de Gantt (Nov - Ene).	22
4.2. Diagrama de Gantt (Ene - Mar).	22
4.3. Diagrama de Gantt (Mar - May).	22
4.4. Diagrama de Gantt (May - Jun).	22
4.5. Evidencia de la ejecución del flujo de integración continua.	25
5.1. Diagrama de arquitectura general del sistema.	28
5.2. Esquema físico de la tabla «cars».	32
5.3. Diagrama de flujo del proceso de normalización e ingesta.	35
5.4. Diagrama de secuencia del pipeline de IA. Vista general.	40
5.5. Diagrama de secuencia del pipeline de IA. Bucle de tool-calling.	41
5.6. Diagrama de secuencia del pipeline de IA. Generación del envelope.	41
5.7. Diagrama de secuencia del pipeline de IA. Hidratación de imágenes.	42
5.8. Interfaz conversacional con respuesta de gráfica.	43
5.9. Interfaz conversacional con respuesta de tabla.	43
5.10. Interfaz conversacional con respuesta de imágenes.	44
5.11. Vista de catálogo con filtros.	44
5.12. Vista de favoritos.	45
5.13. Vista de historial.	46
5.14. Vista del panel de preferencias de accesibilidad.	47
7.1. Informe de cobertura de código del <i>backend</i> generado en la integración continua.	71

ÍNDICE DE TABLAS

3.1. Resumen de los requisitos funcionales y su prioridad.	10
4.1. Principales desviaciones entre la planificación inicial y la implementación final.	20
4.2. Fases del proyecto y fechas límite según la planificación inicial.	21
4.3. Herramientas y tecnologías empleadas, clasificadas por ámbito.	23
5.1. Bloques de atributos de la tabla <i>cars</i>	31
5.2. Vocabulario canónico del atributo <i>combustible</i>	33
5.3. <i>Endpoints</i> de la API REST del <i>backend</i>	36
5.4. Herramientas (<i>tools</i>) disponibles para el modelo de lenguaje.	38
6.1. Ejemplos de saneamiento y unificación de vocabularios: del texto crudo del portal al valor canónico tipado.	53
6.2. Comando de arranque de cada servicio según el destino multietapa (<i>dev/prod</i>) de su <i>Dockerfile</i>	61
7.1. Herramientas de prueba por componente, fronteras aisladas y niveles cubiertos.	65
7.2. Módulos de prueba del <i>backend</i> y su cometido.	66
7.3. Muestra de la evaluación cualitativa del asistente sobre consultas representativas.	73
7.4. Trazabilidad entre requisitos y mecanismos de verificación.	74
A.1. Campos de una entrada en archivo de bibliografía <i>.bib</i>	82
B.1. Tipos posibles de elementos en bibliografía con <i>natbib</i>	84

LISTADOS

6.1. Cascada de <i>callbacks</i> del <i>spider</i> (extracto de <code>km77_spider.py</code>).	50
6.2. Encadenado de <i>crawl</i> e ingesta (<code>run_scrape.sh</code>).	51
6.3. Extracto de <code>_FILTROS_SCHEMA</code> : el enum cerrado del combustible (<code>ai_service.py</code>).	57

ABREVIATURAS

ESI	Escuela Superior de ingeniería
InSo	Ingeniería del Software
SBSE	Search based software engineering
TFG	Trabajo fin de grado
UAL	Universidad de Almería

RESUMEN Y ABSTRACT

En este archivo se incluirá tanto el resumen en castellano como su traducción al inglés. Los dos párrafos estarán ligeramente separados.

El propósito del trabajo en una o dos frases. El diseño y metodología utilizada, los resultados más significativos del trabajo realizado y un breve resumen de las conclusiones. Debe ser conciso y presentar los resultados obtenidos tras la ejecución del TFG.

Put here the english translation Debe ser conciso y presentar los resultados obtenidos tras la ejecución del TFG. Los dos párrafos estarán ligeramente separados.

1 INTRODUCCIÓN

1.1 CONTEXTO Y MOTIVACIÓN

1.2 DESCRIPCIÓN DEL PROBLEMA

1.3 OBJETIVOS DEL TRABAJO

1.3.1 Objetivo general

1.3.2 Objetivos específicos

1.4 ESTRUCTURA DE LA MEMORIA

2 ESTADO DEL ARTE

2.1 BUSCADORES Y AGREGADORES DE COCHES EXISTENTES

2.2 MODELOS DE LENGUAJE Y PROCESAMIENTO DE LENGUAJE NATURAL

2.3 WEB SCRAPING PARA LA OBTENCIÓN DE DATOS

2.4 TECNOLOGÍAS WEB MODERNAS PARA APLICACIONES INTERACTIVAS

2.5 CONCLUSIONES DEL ANÁLISIS

3 ANÁLISIS DE REQUISITOS

Este capítulo recoge el análisis de requisitos del sistema desarrollado, denominado *autoAI*. Define qué debe hacer la aplicación, bajo qué restricciones de calidad ha de operar y cómo se traduce ese comportamiento en las interacciones que ofrece al usuario. Para que el análisis se corresponda con el sistema construido, los requisitos que se enuncian a continuación se han extraído tanto de los objetivos planteados en el Capítulo 1 como de la propia implementación: el catálogo y el asistente conversacional del cliente web, la API REST del backend, el modelo de datos relacional y el proceso de extracción y normalización de datos. Así, cada requisito es trazable a un componente real del sistema.

3.1 DESCRIPCIÓN GENERAL DEL SISTEMA

autoAI es una aplicación web cuyo propósito es ayudar a una persona no experta a encontrar y comparar vehículos del mercado español a partir de la información técnica y comercial publicada por el agregador `km77.com`. El sistema combina dos modos de uso complementarios. El primero es un **catálogo filtrable** convencional, en el que el usuario aplica criterios estructurados (marca, carrocería, combustible, rango de precio, potencia, etc.) y navega los resultados paginados. El segundo es un **asistente conversacional** basado en un modelo de lenguaje, al que el usuario puede dirigirse en lenguaje natural, incluso con peticiones vagas como «un coche para la familia» o «algo que gaste poco», y que responde con texto, tablas, gráficas e imágenes construidas a partir de datos reales de la base de datos.

Desde el punto de vista arquitectónico, el sistema se organiza en cuatro componentes desplegados como contenedores independientes y orquestados con Docker Compose, tal como se describe en el fichero `docker-compose.yml`:

- Un **servicio de extracción** (*scraper*), implementado con Scrapy, que recorre el sitio `km77.com` en cinco niveles (marcas, modelos, submodelos, versiones y ficha técnica de cada versión) y vuelca los datos en bruto.
- Un **backend** desarrollado con FastAPI, que expone una API REST, normaliza e ingiere los datos extraídos, sirve el catálogo y orquesta las llamadas al modelo de lenguaje mediante una arquitectura de *tool calling*.
- Una **base de datos** relacional PostgreSQL, con una única tabla `cars` que almacena las versiones normalizadas.

- Un **frontend** de tipo *single-page application*, desarrollado con React y TypeScript, que ofrece la interfaz de usuario (asistente, catálogo, favoritos, historial y ajustes de accesibilidad).

Un rasgo determinante para el análisis es que el sistema **no dispone de gestión de usuarios ni de autenticación de cuentas**. Toda la información propia del usuario (las conversaciones del historial, la lista de favoritos y las preferencias de accesibilidad) se persiste exclusivamente en el almacenamiento local del navegador (`localStorage`), como reflejan los módulos del cliente. Esta decisión, acorde con el alcance de un Trabajo de Fin de Grado, condiciona varios de los requisitos que se detallan más adelante.

La organización interna de estos componentes, la red que los comunica y los flujos de datos entre ellos se describen en el diseño del sistema (véase la Figura 5.1 del Capítulo 5).

3.2 ACTORES DEL SISTEMA

Al no existir cuentas de usuario, el conjunto de actores es reducido. Se distinguen actores humanos y actores externos de carácter automático.

Usuario final. Persona que interactúa con la aplicación web a través del navegador. No necesita registrarse ni iniciar sesión: cualquier visitante dispone de las mismas capacidades. Es el actor principal y protagoniza la mayoría de los casos de uso (consultar el catálogo, conversar con el asistente, gestionar favoritos e historial y ajustar las preferencias de accesibilidad). No se contempla un rol de usuario avanzado distinto del usuario general.

Administrador / responsable del despliegue. Persona encargada de instalar, configurar y mantener el sistema. No interactúa con una interfaz gráfica de administración, sino a través de la configuración del entorno (variables de entorno, fichero `.env`, Docker Compose) y de los procesos de extracción e ingesta de datos. Es quien provee la clave de la API del modelo de lenguaje y quien puede activar los mecanismos de protección opcionales del backend (limitación de tasa y clave de API para el chat).

Proceso de extracción de datos (*scraper*). Actor automático que, de forma programada, recorre la fuente externa `km77.com`, obtiene los datos en bruto e invoca al backend para su normalización e ingesta. Aunque forma parte del propio sistema, se modela como actor porque actúa de manera autónoma frente a la API.

Proveedor del modelo de lenguaje. Servicio externo (API compatible con la de OpenAI) al que el backend delega la generación de respuestas del asistente. El sistema depende de él para el funcionamiento del modo conversacional, pero no para el catálogo.

Fuente de datos externa (`km77.com`). Sitio web del que se obtiene toda la información de los vehículos. Es un actor pasivo: no ofrece una API, por lo que la obtención de datos se realiza mediante *web scraping*.

3.3 REQUISITOS FUNCIONALES

Los requisitos funcionales describen las capacidades que el sistema debe ofrecer. Se han agrupado por bloques funcionales coincidentes con los subsistemas reales de la aplicación. Cada requisito se identifica con el prefijo **RF** seguido de un número correlativo.

3.3.1 Catálogo y consulta estructurada

- RF-01.** El sistema debe ofrecer un catálogo de vehículos paginado. La paginación se realiza con un tamaño de página fijo y mediante desplazamiento (*limit/offset*), permitiendo avanzar y retroceder entre páginas.
- RF-02.** El sistema debe permitir filtrar el catálogo por, al menos, los siguientes criterios: marca, modelo, tipo de combustible, carrocería, tracción, transmisión, precio mínimo, precio máximo, potencia mínima y número mínimo de plazas. Los filtros deben poder combinarse entre sí.
- RF-03.** El sistema debe proporcionar al cliente los valores disponibles para los filtros categóricos (marcas, combustibles, carrocerías, tracciones y transmisiones presentes en la base de datos), de modo que la interfaz pueda construir los desplegables de filtrado sin valores inexistentes.
- RF-04.** El sistema debe permitir consultar la ficha completa de un vehículo concreto a partir de su identificador, devolviendo todos sus atributos técnicos y comerciales.
- RF-05.** El sistema debe permitir comparar varios vehículos a la vez, recuperando sus fichas completas a partir de una lista de identificadores y preservando el orden solicitado.
- RF-06.** El sistema debe ordenar por defecto los resultados del catálogo mostrando primero las versiones más recientes, de forma determinista para que la paginación sea estable.

3.3.2 Asistente conversacional

- RF-07.** El sistema debe ofrecer un asistente conversacional capaz de interpretar preguntas formuladas en lenguaje natural sobre vehículos y responderlas apoyándose en los datos reales almacenados.
- RF-08.** El asistente debe mantener el contexto de la conversación, admitiendo varios turnos de mensajes de usuario y de respuesta del sistema dentro de una misma sesión.
- RF-09.** El asistente nunca debe inventar datos de los vehículos (precios, potencias, consumos, fechas o identificadores): toda cifra que ofrezca debe proceder de una consulta a la base de datos realizada a través de las herramientas internas (*tools*) de búsqueda, consulta de ficha, comparación y agregación.
- RF-10.** El asistente debe ser capaz de interpretar intenciones vagas o implícitas (por ejemplo, «coche para la familia», «algo deportivo», «que gaste poco» o «para ciudad») traducíéndolas a filtros concretos sobre los atributos disponibles, y de buscar siempre antes de responder.

- RF-11.** El asistente debe estructurar sus respuestas en bloques diferenciados, que pueden ser texto, tablas, gráficas (de barras o radar) e imágenes de vehículos, de manera que la interfaz pueda representarlas de forma rica.
- RF-12.** El asistente debe poder generar visualizaciones (gráficas) a partir de agregaciones (recuento, media, mínimo y máximo) sobre campos numéricos, opcionalmente agrupadas por una dimensión categórica (por ejemplo, precio medio por marca o por carrocería).
- RF-13.** Cuando se solicite un atributo que el sistema no puede filtrar (por ejemplo, techo solar, color, etiqueta medioambiental u otras características no presentes en el modelo de datos), el asistente debe informar de esa limitación, indicar la dimensión soportada más cercana y, aun así, ejecutar la búsqueda con el resto de criterios válidos en lugar de rendirse.
- RF-14.** El asistente debe poder responder en español o en inglés según la preferencia de idioma del usuario, traduciendo únicamente la prosa generada y manteniendo intactos los datos del vehículo (marcas, modelos, especificaciones y valores técnicos).
- RF-15.** Ante una respuesta del modelo que no se ajuste al formato estructurado esperado, el sistema debe reintentar la generación y, si aun así falla, devolver un mensaje de error controlado en lugar de una respuesta inválida.

3.3.3 Datos del usuario en el navegador

- RF-16.** El sistema debe permitir al usuario marcar y desmarcar vehículos como favoritos, y consultar en cualquier momento la lista de sus favoritos, persistida localmente en el navegador.
- RF-17.** El sistema debe guardar automáticamente las conversaciones del usuario como sesiones de historial, permitiéndole consultarlas, reabrir las y eliminarlas, así como iniciar nuevas conversaciones.
- RF-18.** El sistema debe ofrecer ajustes de accesibilidad y preferencia (tema claro u oscuro, tamaño de fuente e idioma de la interfaz) que se apliquen de inmediato y se conserven entre sesiones del navegador.

3.3.4 Extracción, normalización e ingesta de datos

- RF-19.** El sistema debe extraer automáticamente, de la fuente externa km77.com, la información técnica y comercial de los vehículos, recorriendo la jerarquía marca → modelo → submodelo → versión y obteniendo la ficha de cada versión.
- RF-20.** El sistema debe normalizar los datos en bruto a un formato canónico antes de almacenarlos: conversión de unidades y formatos numéricos, interpretación de fechas de comercialización, traducción de las carrocerías a un vocabulario controlado e inferencia del tipo de combustible y nivel de electrificación (gasolina, gasóleo, gas, eléctrico, microhíbrido, híbrido y enchufable) a partir del nombre y del submodelo de la versión.

RF-21. El sistema debe ingerir los datos normalizados en la base de datos de forma idempotente, de modo que reejecutar la extracción actualice las versiones existentes en lugar de duplicarlas (empleando la URL de la versión como clave única).

RF-22. El proceso de ingesta debe ser tolerante a fallos por registro: un dato individual inválido no debe abortar la ingesta del lote, y el sistema debe devolver un resumen del proceso (leídos, validados, normalizados, guardados y errores en cada fase).

RF-23. El sistema debe poder ejecutar la extracción de forma programada y periódica, de acuerdo con una expresión de planificación configurable.

La Tabla 3.1 resume los veintitrés requisitos funcionales y les asigna una prioridad orientativa según su importancia para cumplir los objetivos del trabajo: *alta* para las capacidades nucleares (catálogo, asistente y obtención de datos), *media* para las funciones de apoyo y *baja* para los aspectos opcionales o accesorios.

ID	Descripción breve	Prioridad
RF-01	Catálogo de vehículos paginado (<i>limit/offset</i>).	Alta
RF-02	Filtrado combinable por marca, modelo, combustible, carrocería, etc.	Alta
RF-03	Provisión de los valores disponibles para los filtros categoricos.	Media
RF-04	Consulta de la ficha completa de un vehículo por su identificador.	Alta
RF-05	Comparación de varios vehículos preservando el orden solicitado.	Media
RF-06	Orden por defecto determinista por novedad (paginación estable).	Media
RF-07	Asistente conversacional sobre datos reales.	Alta
RF-08	Mantenimiento del contexto multiturno de la conversación.	Alta
RF-09	Prohibición de inventar datos: toda cifra procede de una <i>tool</i> .	Alta
RF-10	Interpretación de intenciones vagas como filtros concretos.	Alta
RF-11	Respuestas en bloques (texto, tabla, gráfica e imagen).	Alta
RF-12	Generación de gráficas a partir de agregaciones.	Media
RF-13	Gestión de atributos no soportados (aviso, alternativa y búsqueda).	Media
RF-14	Respuesta bilingüe (español/inglés) sin traducir los datos.	Media
RF-15	Reintento y error controlado ante respuestas mal formadas.	Media
RF-16	Gestión de favoritos persistidos en el navegador.	Baja
RF-17	Historial de conversaciones (consultar, reabrir y eliminar).	Baja
RF-18	Ajustes de accesibilidad (tema, fuente e idioma) persistentes.	Media
RF-19	Extracción jerárquica de los datos de <i>km77.com</i> .	Alta
RF-20	Normalización de los datos a un formato canónico.	Alta
RF-21	Ingesta idempotente (URL como clave única).	Alta
RF-22	Ingesta tolerante a fallos por registro, con resumen del proceso.	Media
RF-23	Ejecución programada y periódica de la extracción.	Baja

Tabla 3.1: Resumen de los requisitos funcionales y su prioridad.

3.4 REQUISITOS NO FUNCIONALES

Los requisitos no funcionales establecen las restricciones de calidad bajo las que debe operar el sistema. Se identifican con el prefijo **RNF**.

3.4.1 Usabilidad y accesibilidad

RNF-01. La interfaz debe poder ser utilizada por personas sin conocimientos técnicos de automoción, ofreciendo tanto un acceso guiado por filtros (catálogo) como un acceso libre en lenguaje natural (asistente).

3.4. REQUISITOS NO FUNCIONALES

RNF-02. La aplicación debe ofrecer ajustes de accesibilidad básicos (tema claro/oscuro y tamaño de fuente), y los diálogos modales deben respetar buenas prácticas de accesibilidad: rol de diálogo, atrapado de foco, cierre con la tecla `Esc` y devolución del foco al elemento que los abrió.

RNF-03. La interfaz debe ser multilingüe (español e inglés), tanto en sus textos estáticos como en la prosa generada por el asistente.

RNF-04. La aplicación debe proporcionar retroalimentación visual durante las operaciones asíncronas (esqueletos de carga en el catálogo, indicador de escritura del asistente) y mensajes claros ante estados de error o ausencia de resultados.

RNF-05. La interfaz debe advertir explícitamente de que las respuestas generadas por inteligencia artificial pueden contener errores.

3.4.2 Fiabilidad y robustez

RNF-06. El sistema debe degradar de forma controlada ante datos ausentes: numerosos atributos de los vehículos pueden faltar en la fuente (maletero en *pickups*, número de marchas en eléctricos e híbridos con cambio continuo, etc.), y tanto el modelo de datos como la interfaz deben admitir valores nulos sin producir fallos.

RNF-07. El backend no debe exponer detalles internos en las respuestas de error: los mensajes al cliente deben ser genéricos y el detalle técnico debe quedar registrado únicamente en los *logs*.

RNF-08. El sistema debe ser observable: las operaciones del asistente deben registrar un identificador de petición, las herramientas invocadas y el número de resultados, para facilitar la depuración del comportamiento del modelo.

3.4.3 Seguridad

RNF-09. Todas las consultas a la base de datos deben construirse de forma segura frente a inyección SQL: los valores aportados por el usuario o por el modelo de lenguaje deben pasar siempre parametrizados, y los nombres de métrica, campo y agrupación de las agregaciones deben validarse contra listas blancas antes de interpolarse en la consulta.

RNF-10. El endpoint de ingesta debe impedir el acceso a rutas fuera del directorio autorizado, bloqueando rutas absolutas y travesías de directorios.

RNF-11. El sistema debe ofrecer mecanismos opcionales de protección del backend (limitación de tasa de peticiones por minuto y exigencia de una clave de API para el chat y la ingesta) que puedan activarse mediante configuración sin alterar el comportamiento por defecto.

RNF-12. La clave de acceso al proveedor del modelo de lenguaje y demás credenciales deben gestionarse mediante variables de entorno y nunca codificarse en el código fuente.

3.4.4 Rendimiento

- RNF-13.** Las consultas más frecuentes del catálogo deben estar soportadas por índices en la base de datos (marca, modelo, precio, combustible, carrocería, potencia, plazas y fecha de inicio de comercialización), incluyendo un índice parcial para la consulta de versiones vigentes.
- RNF-14.** El bucle de razonamiento del asistente debe estar acotado en número de iteraciones de llamada a herramientas, para evitar consumos excesivos de tiempo y de coste frente al proveedor del modelo.
- RNF-15.** El proceso de extracción debe respetar la fuente de datos, empleando límites de concurrencia, retardos entre peticiones y regulación automática del rendimiento (*autothrottle*), así como la posibilidad de reanudar trabajos interrumpidos.

3.4.5 Mantenibilidad y despliegue

- RNF-16.** El sistema debe poder desplegarse de forma reproducible mediante contenedores, con cada componente (extracción, backend, base de datos y frontend) aislado y comunicado a través de una red interna.
- RNF-17.** La configuración del sistema (cadena de conexión a la base de datos, orígenes CORS permitidos, modelo de lenguaje, planificación del *scraper*, etc.) debe ser externa al código y parametrizable por entorno.
- RNF-18.** El código debe disponer de pruebas automatizadas tanto en el backend como en el frontend que respalden las funcionalidades críticas de normalización, repositorio de datos y componentes de interfaz.

3.5 CASOS DE USO

A partir de los actores y de los requisitos funcionales identificados, esta sección describe los principales casos de uso del sistema. El actor central es el *usuario final*, que protagoniza la interacción; se incluyen además los casos del actor automático de extracción. Cada caso de uso se identifica con el prefijo **CU**.

3.5.1 CU-01: Explorar el catálogo de vehículos

Actor principal: Usuario final.

Requisitos relacionados: RF-01, RF-02, RF-03, RF-06.

Precondición: La base de datos contiene vehículos ingeridos.

Flujo principal:

1. El usuario accede a la vista de catálogo.
2. El sistema carga los valores disponibles para los filtros y muestra la primera página de resultados, ordenados por novedad.

3. El usuario selecciona uno o varios filtros (marca, combustible, carrocería, rango de precio, etc.) y los aplica.
4. El sistema recupera y muestra los vehículos que cumplen los criterios, paginados.
5. El usuario navega entre páginas o ajusta los filtros.

Flujos alternativos: Si ningún vehículo cumple los criterios, el sistema muestra un estado vacío explicativo. Si se produce un error de comunicación, el sistema muestra un mensaje de error y ofrece reintentar.

Postcondición: El usuario visualiza el subconjunto de vehículos deseado.

3.5.2 CU-02: Consultar y comparar fichas de vehículos

Actor principal: Usuario final.

Requisitos relacionados: RF-04, RF-05.

Precondición: Existen vehículos en el catálogo.

Flujo principal:

1. El usuario selecciona un vehículo para ver su ficha completa.
2. El sistema muestra todos los atributos técnicos y comerciales de la versión.
3. Opcionalmente, el usuario solicita comparar varios vehículos.
4. El sistema presenta sus fichas enfrentadas, respetando el orden indicado.

Flujo alternativo: Si el identificador solicitado no existe, el sistema informa de que el vehículo no se ha encontrado.

Postcondición: El usuario dispone de la información detallada de uno o varios vehículos.

3.5.3 CU-03: Consultar al asistente conversacional

Actor principal: Usuario final.

Actores secundarios: Proveedor del modelo de lenguaje.

Requisitos relacionados: RF-07 a RF-15.

Precondición: El servicio de modelo de lenguaje está configurado y disponible.

Flujo principal:

1. El usuario escribe una pregunta en lenguaje natural sobre vehículos.
2. El sistema interpreta la intención y consulta la base de datos a través de sus herramientas internas (búsqueda, ficha, comparación o agregación).
3. El sistema genera una respuesta estructurada en bloques de texto, tablas, gráficas o imágenes, en el idioma preferido del usuario.
4. El usuario lee la respuesta y, si lo desea, continúa la conversación con nuevos mensajes que el sistema interpreta en contexto.

Flujos alternativos:

- Ante una petición vaga, el sistema infiere filtros razonables y busca igualmente, refinando después.

- Ante un atributo no soportado, el sistema lo advierte, propone la dimensión más cercana y ejecuta la búsqueda con el resto de criterios.
- Si el modelo devuelve una respuesta mal formada, el sistema reintenta y, en última instancia, muestra un mensaje de error controlado.

Postcondición: El usuario obtiene una respuesta basada en datos reales del catálogo.

3.5.4 CU-04: Gestionar la lista de favoritos

Actor principal: Usuario final.

Requisitos relacionados: RF-16.

Precondición: Ninguna.

Flujo principal:

1. El usuario marca un vehículo como favorito.
2. El sistema persiste la elección en el navegador.
3. El usuario consulta la vista de favoritos, donde se muestran las fichas de los vehículos guardados.
4. El usuario puede desmarcar un vehículo, que desaparece de la lista.

Postcondición: La lista de favoritos refleja la selección del usuario y se conserva entre sesiones del navegador.

3.5.5 CU-05: Gestionar el historial de conversaciones

Actor principal: Usuario final.

Requisitos relacionados: RF-17.

Precondición: Ninguna.

Flujo principal:

1. Al conversar, el sistema guarda automáticamente la sesión con un título derivado del primer mensaje del usuario.
2. El usuario accede al historial y visualiza sus conversaciones, ordenadas de la más reciente a la más antigua.
3. El usuario reabre una conversación previa o inicia una nueva.
4. El usuario puede eliminar una conversación del historial.

Postcondición: El historial refleja las conversaciones del usuario, almacenadas localmente.

3.5.6 CU-06: Ajustar las preferencias de accesibilidad

Actor principal: Usuario final.

Requisitos relacionados: RF-18, RNF-02, RNF-03.

Precondición: Ninguna.

Flujo principal:

1. El usuario abre el panel de ajustes.
2. Selecciona el tema (claro u oscuro), el tamaño de fuente y el idioma de la interfaz.
3. El sistema aplica los cambios de inmediato y los persiste.

Postcondición: La interfaz se presenta según las preferencias elegidas, que se mantienen en visitas posteriores.

3.5.7 CU-07: Comparar opciones mediante visualizaciones

Actor principal: Usuario final.

Actores secundarios: Proveedor del modelo de lenguaje.

Requisitos relacionados: RF-12, RF-11.

Precondición: El asistente está disponible.

Flujo principal:

1. El usuario formula una consulta de naturaleza cuantitativa o comparativa (por ejemplo, «precio medio por marca de los SUV»).
2. El sistema calcula la agregación correspondiente sobre la base de datos.
3. El sistema devuelve una gráfica (de barras o radar) acompañada, en su caso, de texto explicativo.

Flujo alternativo: Si los datos son insuficientes para una gráfica fiable, el sistema recurre a una representación en forma de tabla.

Postcondición: El usuario dispone de una visualización que facilita la comparación.

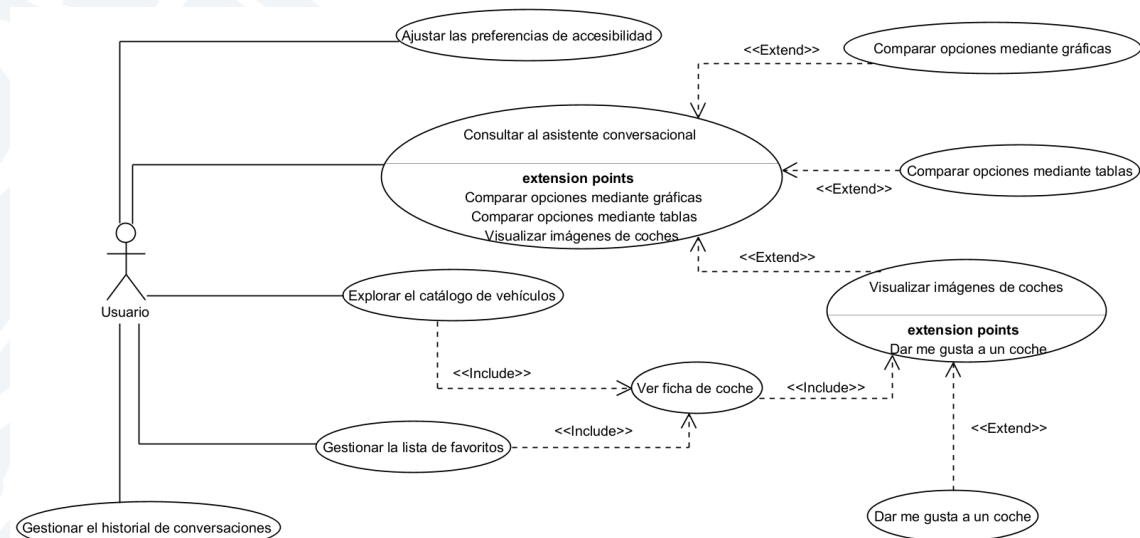


Figura 3.1: Diagrama de casos de uso del usuario final.

4 METODOLOGÍA Y PLANIFICACIÓN

Este capítulo describe cómo se ha desarrollado *autoAI*: la metodología de trabajo adoptada, la planificación temporal en fases y su contraste con la ejecución real, las herramientas y el entorno empleados, y los mecanismos de control de versiones e integración continua. Por tratarse de un Trabajo de Fin de Grado realizado por una sola persona, la metodología se ha adaptado a ese contexto: se han mantenido las prácticas profesionales útiles para un proyecto individual (desarrollo incremental, control de versiones y automatización de las comprobaciones) y se han descartado las que presuponen un equipo, como las reuniones de coordinación o el reparto de roles.

La planificación inicial, recogida en el documento de objetivos elaborado al comienzo del trabajo, no se siguió de forma rígida: a medida que se profundizó en el problema y se construyeron las primeras versiones, varias decisiones técnicas cambiaron. Por ese motivo, además del plan, el capítulo documenta las desviaciones respecto a él y las razones que las motivaron, que forman parte del aprendizaje del proyecto.

4.1 METODOLOGÍA DE DESARROLLO

El desarrollo de *autoAI* ha seguido una metodología incremental e iterativa, de inspiración ágil pero adaptada a un único desarrollador. En lugar de abordar el sistema completo de una vez, se construyó primero un producto mínimo viable (*MVP*) centrado en la funcionalidad nuclear, el asistente conversacional que responde a partir de datos reales. Sobre esa base se añadieron después el catálogo, las visualizaciones, los datos del usuario en el navegador, la internacionalización y la accesibilidad. Cada incremento se desarrolló por completo (implementación, prueba y, cuando hizo falta, refactorización) antes de pasar al siguiente, de modo que en todo momento existiera una versión funcional del sistema.

Esta elección responde a la naturaleza del trabajo. Por ser un proyecto exploratorio, en el que aspectos como la fiabilidad del modelo de lenguaje o la estructura real de los datos de la fuente no se conocían de antemano, una planificación cerrada en cascada habría sido frágil. El enfoque iterativo permitió validar pronto las hipótesis más arriesgadas (si el modelo era capaz de traducir lenguaje natural a filtros fiables, o qué atributos publica realmente [km77.com](https://www.km77.com/)) y corregir el rumbo a partir de lo aprendido, en vez de descubrir los problemas al final.

4.1.1 Principios y prácticas adoptadas

De las metodologías ágiles se han incorporado las prácticas que tienen sentido para un proyecto individual:

Desarrollo guiado por incrementos verticales. Cada nueva capacidad atraviesa todas las capas afectadas del sistema (extracción, base de datos, *backend* y *frontend*) antes de darse por terminada, en lugar de completar una capa entera antes de empezar la siguiente. Así, el catálogo, por ejemplo, se construyó como un incremento completo que va desde la consulta SQL en el repositorio hasta la vista de React.

Refactorización continua. La estructura del código se ha ido mejorando a lo largo del proyecto en lugar de congelarse. El historial de control de versiones recoge numerosas reestructuraciones explícitas, como el cambio de nombre del paquete `app` a `src`, la reorganización de los modelos de datos o la sustitución del proxy directo al modelo de lenguaje por un acceso controlado a través del *backend*.

Pruebas como red de seguridad. Las funcionalidades críticas (normalización de datos, repositorio de coches, validación de las respuestas del asistente y tolerancia a registros malformados en la ingesta) se respaldaron con pruebas automatizadas que permiten refactorizar con confianza (véase el Capítulo 6).

Integración continua. Cada cambio empujado al repositorio dispara una comprobación automática (*lint*, comprobación de tipos, pruebas y construcción de imágenes), de modo que las regresiones se detectan de inmediato y no se acumulan. Esta práctica se detalla en la Sección 4.4.

Mensajes de *commit* estructurados. Para mantener un historial trazable se adoptó la convención *Conventional Commits*, que prefija cada mensaje con su tipo (`feat`, `fix`, `refactor`, `docs`, `build`, `ci`, `test`, `perf`, `chore`). Así, la intención de cada cambio queda explícita y la evolución del proyecto resulta fácil de reconstruir.

4.1.2 Gestión del cambio: desviaciones respecto al plan inicial

El plan inicial se entendió como una guía y no como un contrato. Al avanzar el trabajo, varias decisiones técnicas recogidas en el documento de objetivos se revisaron a la luz de lo aprendido. Las desviaciones más relevantes fueron:

Sistema gestor de base de datos: de MySQL a PostgreSQL. El documento inicial preveía MySQL. Se optó por PostgreSQL por su soporte de índices parciales, empleados para la consulta de versiones vigentes, y por su mejor manejo de valores nulos y tipos, que encaja con un dominio en el que muchos atributos de los vehículos pueden faltar en la fuente.

Automatización del *scraping*: de *n8n* a Scrapy programado. El plan contemplaba orquestar la extracción con la herramienta *n8n*. En su lugar se implementó un *spider* con Scrapy, ejecutado de forma periódica mediante una expresión *cron* dentro del propio contenedor. Esta decisión eliminó una dependencia externa pesada y permitió aprovechar los mecanismos nativos de Scrapy (control de concurrencia, *auto-throttle*, reanudación de trabajos), más apropiados para un *scraping* responsable que un flujo de *n8n*.

Número de fuentes de datos: de varias a una sola. El objetivo inicial hablaba de *scraping* de varias webs. El trabajo se concentró en una única fuente, `km77.com`, por ser la

referencia española en información técnica de automóviles y ofrecer datos completos y homogéneos. Limitar el número de fuentes permitió invertir el esfuerzo en la calidad de la normalización en vez de en la integración de orígenes heterogéneos.

Modelo de datos: de un esquema con opiniones a una única tabla normalizada. El boceto inicial de base de datos incluía tablas de *marca*, *combustible* y *opinion* (reseñas de usuarios) relacionadas con la tabla de coches. En la implementación, el modelo se simplificó a una sola tabla *cars* con las versiones ya normalizadas, y se descartó la recolección de opiniones. El motivo fue doble: las opiniones eran datos de calidad y disponibilidad inciertas que no resultaban esenciales para el sistema, y una tabla desnormalizada simplifica las consultas que el modelo de lenguaje genera a través de sus herramientas.

Alcance de la interacción: hacia un asistente conversacional con *tool calling*. El planteamiento inicial describía un flujo lineal (traducir la consulta a filtros, consultar la base de datos y redactar una respuesta). La implementación derivó hacia una arquitectura de *tool calling*, en la que el modelo decide en cada iteración qué herramientas invocar (búsqueda, ficha, comparación o agregación) hasta reunir datos suficientes para responder. Este cambio, surgido de la experimentación, mejoró la calidad y la fiabilidad de las respuestas.

La Tabla 4.1 resume estas desviaciones, con la decisión recogida en el documento de objetivos, la finalmente implementada y su motivo principal.

Aspecto	Plan inicial	Implementación	Motivo del cambio
Sistema gestor de base de datos	MySQL	PostgreSQL	Índices parciales para versiones vigentes y mejor manejo de nulos y tipos.
Automatización del <i>scraping</i>	n8n	Scrapy con <i>cron</i> en contenedor	Elimina una dependencia externa y aprovecha el control de carga y la reanudación nativos de Scrapy.
Fuentes de datos	Varias webs	Una sola (km77.com)	Concentrar el esfuerzo en la calidad de la normalización sobre datos homogéneos.
Modelo de datos	Tablas marca, combustible y opinion	Tabla única cars desnormalizada	Las opiniones no eran esenciales y una tabla única simplifica las consultas del modelo.
Interacción con el modelo	Flujo lineal (traer, consultar, redactar)	<i>Tool calling</i> iterativo	El modelo decide qué herramientas invocar, mejorando calidad y fiabilidad.

Tabla 4.1: Principales desviaciones entre la planificación inicial y la implementación final.

4.2 PLANIFICACIÓN TEMPORAL Y FASES DEL PROYECTO

La planificación se estructuró en siete fases secuenciales, cada una con sus objetivos y una fecha límite orientativa. La Fase 0 se dedicó a la planificación (definición del objetivo, software, herramientas, boceto de interfaz y esquema preliminar de datos); las siguientes recorren el recorrido del dato y del producto: obtención y limpieza de datos (Fase 1), base de datos (Fase 2), inteligencia artificial y análisis (Fase 3), interfaz de usuario (Fase 4), mejoras y despliegue (Fase 5) y, por último, redacción de la memoria y preparación de la defensa (Fase 6).

La Tabla 4.2 resume las fases, sus tareas principales y las fechas límite previstas en la planificación inicial.



4.2. PLANIFICACIÓN TEMPORAL Y FASES DEL PROYECTO

Fase	Tareas principales	Fecha límite
Fase 0	Planificación: objetivo, software, herramientas, boceto de interfaz y de base de datos.	10/12/2025
Fase 1	Recopilación y limpieza de datos: elección de fuentes, <i>scraping</i> , normalización y pruebas.	24/12/2025
Fase 2	Base de datos: diseño, conexión con la aplicación, almacenamiento de datos normalizados y pruebas.	10/01/2026
Fase 3	IA y análisis: consulta de datos, modelos de recomendación, entrada de consultas y pruebas.	03/02/2026
Fase 4	Interfaz de usuario: interfaz web funcional conectada al <i>backend</i> y pruebas.	05/03/2026
Fase 5	Mejoras: automatización del <i>scraping</i> , gráficas y tablas, contenerización, despliegue y pruebas completas.	15/04/2026
Fase 6	Memoria y defensa: redacción de la memoria y preparación de la presentación.	18/05/2026

Tabla 4.2: Fases del proyecto y fechas límite según la planificación inicial.

4.2.1 Plan previsto frente a ejecución real

La ejecución real respetó el orden global de las fases pero no su calendario exacto. El enfoque iterativo provocó que algunas fases se solaparan en el tiempo en lugar de sucederse de forma estanca, y que el grueso del trabajo se concentrara entre enero y junio de 2026. Las observaciones más destacables son:

- La **planificación** (Fase 0) y la primera puesta en marcha del repositorio se desarrollaron entre el 20 de noviembre y el 15 de diciembre de 2025, con la definición del proyecto y la inicialización de los esqueletos de *backend* y *frontend*.
- La **obtención y normalización de datos** (Fase 1) se prolongó desde el 17 de enero hasta el 14 de abril de 2026 y fue la fase más extensa del proyecto. En ese periodo se construyó el entorno de Scrapy y el *spider*, se mejoró la cantidad y la calidad de los datos extraídos, se añadió el análisis de variabilidad entre versiones y se implementó la normalización (combustibles, electrificación, capacidad de los eléctricos, etc.). Su duración, mayor de lo previsto, se explica por la complejidad real de los datos de la fuente.
- La **base de datos** (Fase 2), entre el 25 de marzo y el 19 de abril de 2026, se solapó con el tramo final de la fase de datos. En ella se diseñó el esquema, se conectó la aplicación, se automatizó el *scraper* y se habilitó el envío de los datos normalizados a la base de datos.
- La **interfaz de usuario** (Fase 4) comenzó el 20 de abril de 2026 con una primera versión funcional del cliente y se extendió hasta el 9 de mayo, por delante en parte de la fase de inteligencia artificial.

- La **inteligencia artificial** (Fase 3), entre el 4 de mayo y el 12 de junio de 2026, se entrelazó con el final de la fase de interfaz. En ella se construyó el *pipeline* de consultas con el modelo de lenguaje y se sucedió un periodo intenso de refinamiento (variación de respuestas, tratamiento de híbridos, agrupación de carrocerías, ajuste del *prompt*, etc.).
- Las **mejoras** (Fase 5), entre el 30 de mayo y el 20 de junio de 2026, abarcaron un amplio conjunto de tareas: endurecimiento de la seguridad (restricción de rutas en la ingesta, limitación de tasa y clave de API opcionales, eliminación del proxy inseguro al modelo), pruebas de *backend* y *frontend*, integración continua, rediseño de la interfaz, internacionalización, accesibilidad y mejora de las gráficas.
- La **memoria** (Fase 6) se redactó en la fase final del proyecto, entre el 17 y el 24 de junio de 2026.

El desplazamiento del calendario respecto al plan inicial se debió a dos causas: la fase de datos resultó más laboriosa de lo estimado y el componente de inteligencia artificial, por su carácter exploratorio, exigió varias iteraciones de ajuste que no eran previsibles al planificar. Este reajuste no fue un fallo de planificación, sino una muestra del valor de la metodología iterativa, que permitió absorber la incertidumbre sin comprometer el resultado.

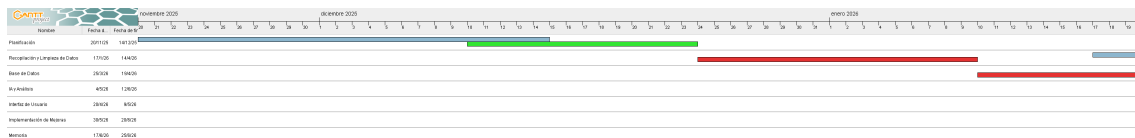


Figura 4.1: Diagrama de Gantt (Nov - Ene).

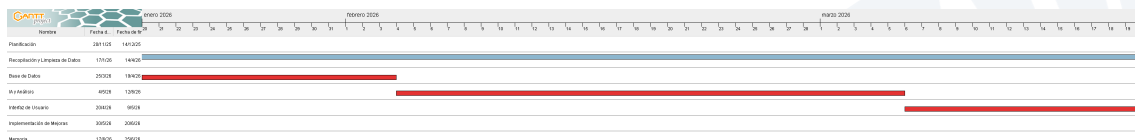


Figura 4.2: Diagrama de Gantt (Ene - Mar).

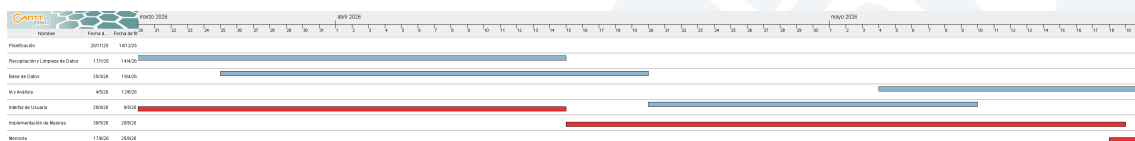


Figura 4.3: Diagrama de Gantt (Mar - May).

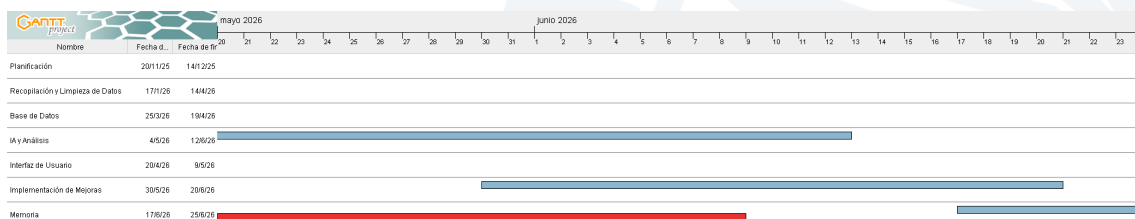


Figura 4.4: Diagrama de Gantt (May - Jun).

4.3 HERRAMIENTAS Y ENTORNO DE TRABAJO

El proyecto se ha desarrollado con herramientas libres y de uso extendido, elegidas por su madurez y su buena integración entre sí. La Tabla 4.3 las clasifica por su función dentro del proyecto.

Ámbito	Herramienta / tecnología
Editor de código	Visual Studio Code
Control de versiones	Git, con repositorio remoto alojado en GitHub
Lenguajes	Python (<i>backend</i> y <i>scraping</i>), TypeScript (<i>frontend</i>)
<i>Backend</i>	FastAPI sobre Python 3.12
<i>Frontend</i>	React 19 con TypeScript, empaquetado con Vite
Base de datos	PostgreSQL 16
Gestión de base de datos	DBeaver (cliente SQL universal)
Extracción de datos	Scrapy
Programación periódica	<i>supercronic</i> (ejecutor de <i>cron</i> en contenedor)
Contenerización	Docker y Docker Compose
Servidor de estáticos	Nginx (sirve la SPA en producción)
Modelo de lenguaje	API compatible con la de OpenAI
Calidad de código (<i>backend</i>)	Ruff (<i>lint</i> y formato), Mypy (tipos)
Calidad de código (<i>frontend</i>)	ESLint, comprobación de tipos con <code>tsc</code>
Pruebas	Pytest (<i>backend</i>), Vitest y Testing Library (<i>frontend</i>)
Integración continua	GitHub Actions
Redacción de la memoria	L ^A T _E X

Tabla 4.3: Herramientas y tecnologías empleadas, clasificadas por ámbito.

Respecto a las herramientas previstas en el documento inicial, el entorno se mantuvo en lo esencial (Visual Studio Code como editor, Git y GitHub para el control de versiones, y Docker para la contenerización), pero se sustituyeron las herramientas ligadas a las decisiones técnicas que evolucionaron: desaparecieron *n8n* y *MySQL Workbench*, al cambiar la automatización del *scraping* y el sistema gestor de base de datos, y se incorporaron las herramientas de calidad (Ruff, Mypy, ESLint), de pruebas (Pytest, Vitest) y de integración continua que no figuraban en el plan original.

El entorno de desarrollo se ha definido también con contenedores, de modo que la máquina del desarrollador solo necesita Docker para levantar el sistema completo. La configuración de Docker Compose distingue dos perfiles, desarrollo y producción, mediante ficheros de superposición (`docker-compose.override.yml` y `docker-compose.prod.yml`). El perfil de desarrollo monta el código fuente como volumen para habilitar la recarga en caliente y expone los puertos de cada servicio; el de producción construye imágenes optimizadas y delega el servicio de los estáticos en Nginx. Los detalles de esta infraestructura se exponen en el Capítulo 6.

4.4 CONTROL DE VERSIONES E INTEGRACIÓN CONTINUA

El proyecto se ha gestionado con Git y un repositorio remoto en GitHub, que ha servido a la vez como copia de seguridad, registro trazable de la evolución del trabajo y disparador de la integración continua.

4.4.1 Estrategia de control de versiones

El flujo de trabajo combinó el desarrollo sobre la rama principal con el uso de ramas de funcionalidad para los bloques de trabajo más voluminosos, que se integraron mediante *pull requests*. El historial recoge, por ejemplo, ramas dedicadas al *scraping* y al *frontend* fusionadas a la rama principal cuando alcanzaron un estado estable. Sobre la rama principal se realizaron, además, *commits* incrementales para correcciones, pruebas y mejoras menores.

Los mensajes de *commit* siguen la convención *Conventional Commits*, que da al historial una semántica uniforme y permite distinguir las nuevas funcionalidades (*feat*) de las correcciones (*fix*), las refactorizaciones (*refactor*), los cambios de construcción o infraestructura (*build*, *ci*, *chore*), la documentación (*docs*), las pruebas (*test*) y las mejoras de rendimiento (*perf*).

4.4.2 Integración continua con GitHub Actions

Para que el código del repositorio se mantenga en buen estado y los errores no se acumulen, se configuró un flujo de integración continua con GitHub Actions (archivo `.github/workflows/ci.yml`) que se ejecuta de forma automática en cada *push* a cualquier rama. El flujo se compone de tres trabajos:

Trabajo de *backend*. Sobre Python 3.12, instala las dependencias y ejecuta, en este orden: el análisis estático con Ruff (que bloquea la construcción ante errores de *lint*), la comprobación de tipos con Mypy en modo gradual (informativa, no bloqueante) y la batería de pruebas con Pytest junto con la medición de cobertura. Para que las pruebas de integración dispongan de una base de datos real, el trabajo levanta un *service container* con PostgreSQL 16 y le apunta la cadena de conexión. El informe de cobertura se publica como artefacto del flujo.

Trabajo de *frontend*. Sobre Node.js 22, instala las dependencias de forma reproducible (`npm ci`) y ejecuta el *linter* (ESLint), la construcción del proyecto (`tsc` más Vite, que incluye la comprobación de tipos de TypeScript) y las pruebas con Vitest.

Trabajo de Docker. Supeditado a que los dos trabajos anteriores hayan concluido con éxito, verifica que las imágenes de contenedor del *backend* y del *frontend* se construyen correctamente, sin publicarlas, lo que confirma que el sistema sigue siendo desplegable.

Así, cada cambio queda validado de forma automática antes de consolidarse, tanto en la calidad y corrección del código como en su comportamiento frente a las pruebas y en su capacidad de despliegue. Esta validación funciona como red de seguridad en un proyecto desarrollado de forma incremental por una sola persona.

4.4. CONTROL DE VERSIONES E INTEGRACIÓN CONTINUA

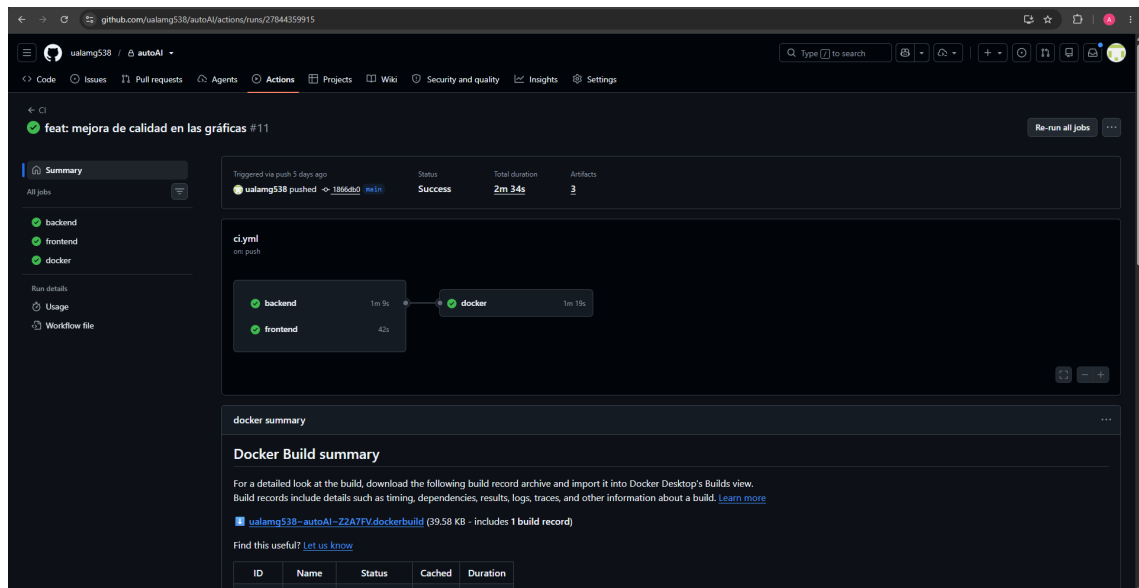


Figura 4.5: Evidencia de la ejecución del flujo de integración continua.

5 DISEÑO DEL SISTEMA

En este capítulo se describe el diseño de *autoAI* en un nivel intermedio de abstracción: a partir de los requisitos recogidos en el capítulo anterior, se definen la arquitectura global del sistema, la estructura de los datos que lo sustentan, los servicios que lo componen y los flujos de información que los conectan. El objetivo es justificar las decisiones de diseño antes de descender, en el capítulo de implementación, al detalle del código. La exposición recorre las capas del sistema: primero la visión de conjunto y el reparto de responsabilidades entre componentes; después el modelo de datos y su representación en la base de datos; a continuación el diseño del *backend* y de su API REST; seguidamente el pipeline que traduce una consulta en lenguaje natural en una respuesta fundamentada en datos reales; y, por último, el diseño de la interfaz de usuario.

5.1 ARQUITECTURA GENERAL

autoAI se estructura como un sistema distribuido de cuatro **servicios desacoplados**, cada uno con una única responsabilidad y empaquetado en su propio contenedor. Esta separación responde a la heterogeneidad de las tareas del sistema: extraer datos de una fuente web externa, almacenarlos de forma estructurada, exponerlos a través de una API, razonar sobre ellos mediante un modelo de lenguaje y presentarlos en una interfaz interactiva. Cada una de estas actividades tiene un ciclo de vida, unas dependencias tecnológicas y un patrón de ejecución distintos, por lo que aislarlas en componentes separados facilita el mantenimiento, permite escalarlas o sustituirlas por separado y evita que un fallo en una comprometa a las demás.

Los cuatro servicios son los siguientes:

- **Base de datos** (bd): una instancia de *PostgreSQL* que actúa como única fuente de verdad del sistema. Almacena el catálogo de vehículos ya normalizado y es el único componente con estado persistente.
- **Backend** (backend): un servicio construido con *FastAPI* (Python) que centraliza toda la lógica de negocio. Expone la API REST que consume la interfaz, contiene el proceso de normalización y carga de datos, y orquesta el pipeline de inteligencia artificial. Es el único componente que se comunica con la base de datos y con el proveedor externo del modelo de lenguaje.
- **Servicio de scraping** (scraper): un proceso basado en *Scrapy* que extrae periódicamente los datos de los vehículos del portal *km77.com*. No accede directamente a la

base de datos: vuelca el resultado en un archivo y solicita su ingesta al *backend*, que es quien lo valida, normaliza y persiste.

- **Frontend** (*frontend*): una aplicación web de página única (SPA) desarrollada con *React* y servida en producción por un servidor *nginx*. Implementa tanto el asistente conversacional como el catálogo navegable, y consume exclusivamente la API REST del *backend*.

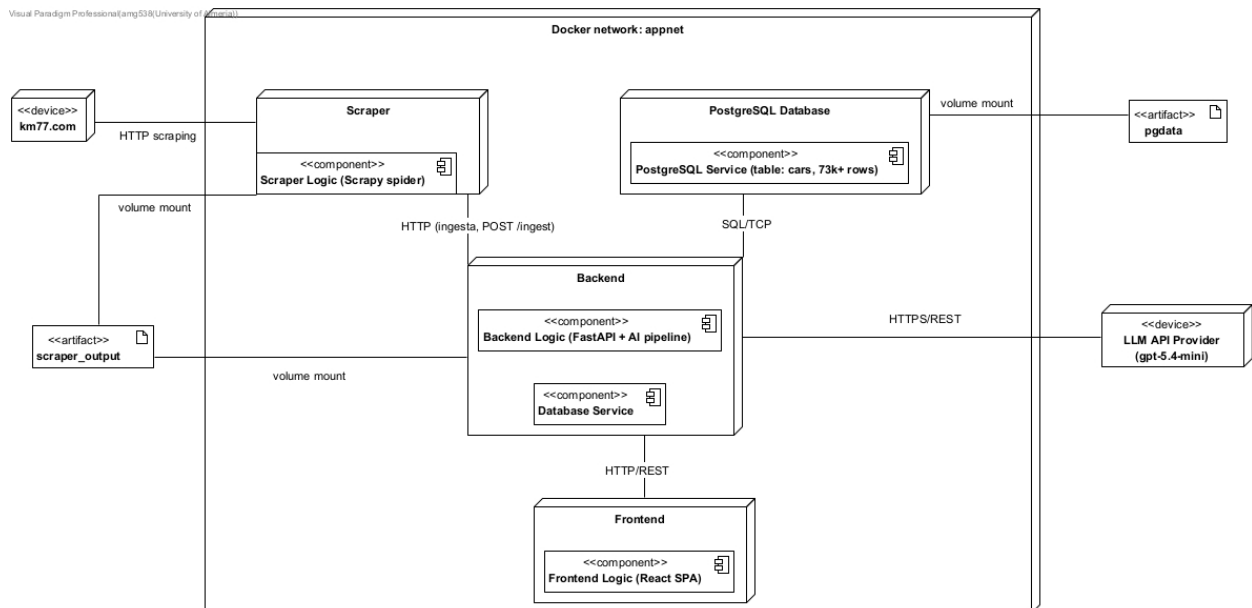


Figura 5.1: Diagrama de arquitectura general del sistema.

La arquitectura se apoya en un **punto de acceso único a los datos**: el *backend* es el único servicio autorizado a leer y escribir en la base de datos. El *frontend* nunca consulta la base de datos directamente, sino que pasa siempre por la API; y el *scraper*, aun produciendo los datos, tampoco los inserta por su cuenta, sino que delega su ingesta en el *backend* mediante una llamada HTTP. Esta decisión concentra en un solo lugar la validación, la normalización y el control de acceso, evita duplicar la lógica de persistencia y garantiza que ningún dato entre en la base de datos sin haber pasado antes por el mismo proceso de saneamiento.

Comunicación entre servicios

Los servicios se comunican mediante dos mecanismos complementarios. El principal es **HTTP/REST**: la interfaz consume los *endpoints* del *backend* para listar el catálogo, comparar vehículos y conversar con el asistente, y el *scraper* invoca el *endpoint* de ingesta cuando termina una extracción. El secundario es un **volumen de disco compartido** entre el *scraper* y el *backend*: el primero escribe en él el archivo con los datos extraídos y el segundo lo lee durante la ingesta. Este reparto evita que el *scraper* necesite conocer el esquema de la base de datos o las credenciales de acceso, y mantiene la frontera entre «producir datos crudos» y «validar y persistir datos» nítidamente trazada.

Todos los servicios residen en una misma red privada virtual y se descubren entre sí por su nombre lógico (bd, backend, etc.), sin necesidad de direcciones fijas. El arranque está ordenado según las dependencias reales: el *backend* no se inicia hasta que la base de datos responde a su comprobación de salud, y ni el *frontend* ni el *scraper* arrancan hasta que el *backend* está operativo. Con ello se evitan los errores transitorios de arranque sin recurrir a esperas arbitrarias. Los detalles de contenerización y orquestación se describen en el capítulo de implementación.

Patrón arquitectónico interno del *backend*

Dentro del *backend*, la organización del código sigue una **arquitectura en capas** que separa la exposición HTTP de la lógica de dominio y del acceso a datos. Se distinguen cuatro niveles:

- La capa de **API** (paquete `api`) define los *endpoints* REST, valida la entrada y traduce las peticiones HTTP en llamadas a la lógica de negocio. No contiene reglas de dominio ni SQL.
- La capa de **servicios** (paquete `services`) concentra la lógica de negocio: la normalización de los datos, el acceso a la base de datos (a través de un *repositorio*) y la integración con el modelo de lenguaje.
- La capa de **modelos** (paquete `models`) define las estructuras de datos con las que trabaja el sistema mediante *Pydantic*, lo que aporta validación automática y una frontera de tipos explícita.
- La capa **núcleo** (paquete `core`) reúne la configuración transversal y la gestión de la conexión a la base de datos.

Esta separación facilita las pruebas (cada capa puede probarse de forma aislada), localiza los cambios (una modificación en el esquema de la base de datos afecta al repositorio, no a los *endpoints*) y mantiene las dependencias dirigidas en un único sentido: la API depende de los servicios y estos de los modelos y el núcleo, nunca a la inversa.

5.2 MODELO DE DATOS

El modelo de datos de *autoAI* tiene un concepto central: la **versión de un vehículo**. El dominio de los automóviles es jerárquico —una *marca* agrupa varios *modelos*, cada modelo se ofrece en distintos *submodelos* o generaciones, y cada submodelo incluye varias *versiones* o acabados con motorizaciones y equipamientos diferentes—, pero es la versión la que constituye la unidad mínima con significado para el usuario: es a ese nivel donde están definidos el precio, la potencia, el consumo y las dimensiones. Por ello, la versión es la entidad sobre la que se construyen las búsquedas, las comparaciones y las recomendaciones.

Esta entidad se representa en el sistema mediante dos estructuras distintas, que corresponden a dos momentos del ciclo de vida del dato:

- Una representación **cruda** (CocheScrap), que recoge los datos tal y como se extraen del portal de origen. Todos sus campos son cadenas de texto, porque así llegan del *scraping*: incluyen unidades («177 CV / 130 kW»), separadores de miles, símbolos de moneda, rangos de fechas y marcadores de dato ausente. Esta representación es fiel a la fuente, pero inservible para consultar o calcular.
- Una representación **normalizada** (Version), que es el modelo canónico del sistema. Cada atributo tiene ya su tipo correcto (numérico, fecha, texto), las unidades se han eliminado, los vocabularios se han unificado y la ausencia de dato se representa de forma explícita. Es la estructura común a todas las capas: la que devuelve la base de datos, la que consume el pipeline de IA y la que recibe la interfaz.

El paso de la primera representación a la segunda es el proceso de normalización, descrito en la sección 5.2.2. Junto a estas dos estructuras existe una tercera familia de modelos, los **bloques de respuesta** del asistente (Block), que no describen vehículos sino el contenido estructurado que el modelo de lenguaje genera (texto, tablas, gráficas e imágenes); se detallan en la sección 5.4.

5.2.1 Diseño de la base de datos

La persistencia se resuelve con una base de datos relacional *PostgreSQL*. El catálogo se almacena en una **única tabla**, *cars*, **desnormalizada de forma deliberada**, en la que cada fila representa una versión completa de un vehículo con todos sus atributos. La jerarquía marca → modelo → submodelo → versión no se modela con tablas separadas y claves foráneas, sino que se aplanan en cuatro columnas de texto dentro de la misma fila (*marca*, *modelo*, *submodelo*, *nombre*).

Esta elección, que se aparta de la tercera forma normal que cabría esperar en un diseño relacional ortodoxo, está justificada por la naturaleza del problema:

- **La carga de trabajo es de solo lectura.** El usuario nunca edita el catálogo; este se regenera por completo desde la fuente. No existe, por tanto, el riesgo de anomalías de actualización que la normalización busca prevenir, que es la principal razón de ser de esta.
- **El patrón de acceso dominante es el filtrado multicriterio sobre una sola entidad.** Casi todas las consultas (tanto del catálogo como de las herramientas de la IA) seleccionan versiones según combinaciones de atributos de la propia versión. Una tabla plana resuelve estas consultas sin ninguna operación de *join*, lo que simplifica el *repositorio* y mejora el rendimiento.
- **Reduce la complejidad.** Para un catálogo de tamaño moderado y que cabe en memoria, un esquema en estrella o totalmente normalizado añadiría complejidad sin un beneficio práctico apreciable.

Estructura de la tabla *cars*

La tabla *cars* reúne veintiséis columnas que se pueden agrupar conceptualmente en cinco bloques, recogidos en la tabla 5.1. La clave primaria es un identificador entero auto-

incremental (*id*), que es el que el sistema usa internamente para referirse a cada vehículo —en las comparaciones, en la recuperación de fichas y en los bloques de imagen generados por la IA—. Además, la columna *url* (la dirección de la ficha del vehículo en el portal de origen) está declarada **UNIQUE** y desempeña el papel de **clave de negocio**: identifica de forma estable una misma versión entre distintas ejecuciones del *scraping*, lo que permite reconocer si un vehículo ya existe y actualizarlo en lugar de duplicarlo.

Bloque		Columnas principales
Identificación		<i>id</i> (PK), <i>marca</i> , <i>modelo</i> , <i>submodelo</i> , <i>nombre</i> , <i>url</i> (única), <i>foto_url</i>
Vigencia		<i>fecha_inicio</i> , <i>fecha_fin</i> (inicio y fin de comercialización)
Precio		<i>precio</i> (en euros)
Carrocería y dimensiones	y	<i>carroceria</i> , <i>puertas</i> , <i>plazas</i> , <i>longitud</i> , <i>anchura</i> , <i>altura</i> , <i>capacidad_maletero</i>
Mecánica y prestaciones	y	<i>combustible</i> , <i>potencia</i> , <i>aceleracion</i> , <i>velocidad_maxima</i> , <i>peso</i> , <i>consumo_medio</i> , <i>traccion</i> , <i>transmision</i> , <i>numero_marchas</i> , <i>capacidad_deposito</i>

Tabla 5.1: Bloques de atributos de la tabla *cars*.

Los tipos de cada columna se han elegido para reflejar la semántica del dato y no su forma de origen. Las magnitudes físicas se almacenan como valores numéricos con la precisión adecuada (por ejemplo, el precio con dos decimales y las dimensiones con un decimal en milímetros), las fechas de comercialización como tipo **DATE**, los pequeños enteros (*puertas*, *plazas*, *número de marchas*) como enteros cortos, y los atributos categóricos (*combustible*, *carrocería*, *tracción*, *transmisión*) como cadenas que contienen siempre un valor del vocabulario canónico definido en la normalización.

Tratamiento de los datos ausentes

El diseño admite **valores nulos** de forma explícita en la mayoría de las columnas de atributos. La fuente de datos no es uniforme: hay vehículos sin maletero (como los *pick-up*), motorizaciones híbridas o eléctricas sin un número discreto de marchas, y campos que el portal no registra. Modelar la ausencia con **NULL** —en lugar de con valores centinela como un cero o una cadena vacía— permite distinguir «este coche pesa cero», que sería un dato erróneo, de «no se conoce el peso de este coche», que es la situación real. Esta distinción se propaga a todo el sistema: el *backend* ordena los resultados colocando siempre los valores nulos al final, y la interfaz los muestra como «dato no disponible» en lugar de inventar una cifra.

Un caso particular es la **vigencia**, modelada con el par de fechas *fecha_inicio* y *fecha_fin*. La convención es que una *fecha_fin* nula significa que la versión *sigue a la venta hoy*; es decir, el valor nulo no representa aquí un dato desconocido, sino un estado del mundo (vehículo aún en comercialización). Esta semántica es la que permite ofrecer al usuario, por defecto, el mercado actual y no versiones descatalogadas, y es un punto que tanto el *backend* como las instrucciones del modelo de lenguaje tratan de forma explícita.

Índices

Para sostener el rendimiento de las consultas más frecuentes sin recurrir a optimizaciones prematuras, el esquema define un conjunto reducido de índices. Se indexan las columnas sobre las que más habitualmente se filtra (marca, modelo, precio, combustible, carrocería, potencia, plazas) y la columna que gobierna el orden por defecto del catálogo (fecha_inicio, ya que las novedades se muestran primero). Se añade además un **índice parcial** sobre las versiones vigentes —las que tienen fecha_fin nula—: al restringir el índice a las filas que cumplen esa condición, se acelera la consulta «solo vehículos a la venta», una de las más recurrentes, y se ocupa mucho menos espacio que con un índice completo.

Visual Paradigm Professional(amg538(University of Almería))

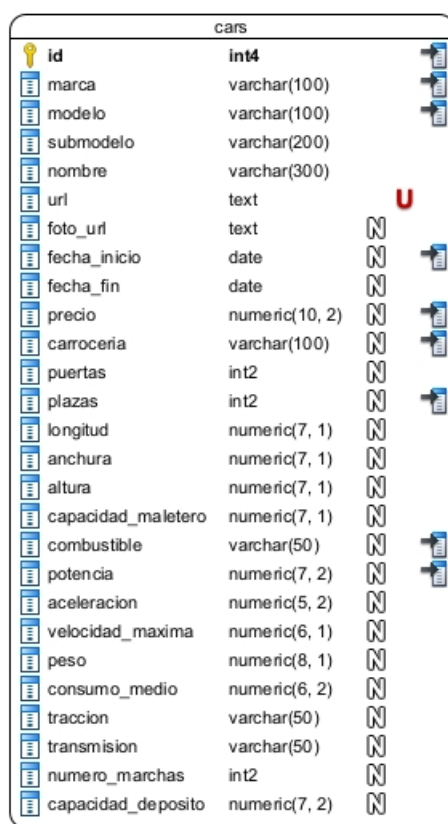


Figura 5.2: Esquema físico de la tabla «cars».

5.2.2 Normalización de los datos de los vehículos

La normalización es una de las partes más delicadas del diseño. Los datos que llegan del *scraping* son fieles a la fuente pero heterogéneos: vienen como texto con unidades incrustadas, con formatos de número propios del español, con rangos de fechas escritos como cadenas y, sobre todo, con vocabularios inconsistentes y datos implícitos. La normalización convierte esos datos en el modelo canónico *Version*, de forma que el resto del sistema —y en particular el modelo de lenguaje— pueda operar sobre datos tipados y con un vocabula-

rio cerrado y predecible. Comprende tres tareas: el saneamiento de los valores escalares, la unificación de los vocabularios categóricos y la persistencia idempotente.

Saneamiento de valores escalares

La parte más mecánica consiste en extraer de cada cadena el valor numérico o la fecha que contiene, descartando todo lo accesorio. Se aplican reglas específicas por tipo de campo: de «177 CV / 130 kW» se conserva únicamente la cifra en caballos; de «4.165 mm» se obtiene el valor en milímetros eliminando el separador de miles; de «7,2 l/100 km» se extrae el número cambiando la coma decimal por punto; y los rangos de fechas del tipo «(03/2021 - 09/2023)» se descomponen en las dos fechas que delimitan la vigencia. Un principio transversal gobierna todo este proceso: ante un valor ausente o un marcador de dato faltante (el portal usa un guion), la conversión **nunca falla ni inventa**, sino que produce un valor nulo. De este modo, una sola versión con un campo problemático no aborta la importación del lote completo.

Unificación de vocabularios categóricos

La dificultad no está en los números, sino en los atributos categóricos, y en particular en el **tipo de combustible**. El portal de origen no clasifica los vehículos por su grado de electrificación: en su campo de combustible solo indica la fuente del motor de combustión (gasolina, gasóleo o combinaciones con gas), y deja ese campo vacío para los eléctricos puros. La información de si un coche es híbrido suave, híbrido autorrecargable o híbrido enchufable no está en ese campo, sino *implícita* en el nombre comercial de la versión.

Para resolverlo se define un **vocabulario canónico cerrado de nueve valores** que codifica, en un único atributo, tanto la fuente fósil como el nivel de electrificación (tabla 5.2). La normalización del combustible determina, por un lado, la base fósil a partir del campo de origen y, por otro, el nivel de electrificación analizando el texto del nombre y el submodelo con expresiones regulares que reconocen los distintos términos comerciales, aplicadas en orden de mayor a menor especificidad (enchufable antes que autorrecargable, y este antes que suave) para evitar clasificaciones erróneas. Este vocabulario es el mismo que después se ofrece al modelo de lenguaje como conjunto cerrado de valores válidos: unificarlo aquí es lo que permite que la IA filtre con precisión más adelante.

Valor canónico	Significado
gasolina	Gasolina pura, sin electrificar
gasoleo	Diésel puro, sin electrificar
gas	<i>Bi-fuel/flex</i> : gasolina con GLP, gas natural o etanol
electrico	100 % eléctrico (BEV)
mhev_gasolina	Microhíbrido (<i>mild hybrid</i> 48 V) de gasolina
mhev_gasoleo	Microhíbrido (<i>mild hybrid</i> 48 V) de diésel
hev_gasolina	Híbrido autorrecargable (<i>full hybrid</i>) de gasolina
phev_gasolina	Híbrido enchufable (PHEV) de gasolina
phev_gasoleo	Híbrido enchufable (PHEV) de diésel

Tabla 5.2: Vocabulario canónico del atributo combustible.

Un tratamiento análogo, aunque más sencillo, recibe el **tipo de carrocería**. El portal emplea sus propias denominaciones (Turismo, SUV/Todoterreno, Turismo familiar, Monovo-

lumen, Descapotable, Pick Up, etc.), que se traducen a un vocabulario canónico de nueve categorías (berlina, suv, compacto, familiar, coupe, cabrio, monovolumen, pickup, furgoneta) mediante una combinación de correspondencias exactas y reglas por subcadena que absorben además sinónimos habituales (*crossover*, *ranchera*, MPV...). El resto de atributos categóricos (tracción y transmisión) se canonizan simplemente pasándolos a minúsculas.

Idempotencia

Las funciones de normalización de vocabulario cumplen la propiedad de **idempotencia**: aplicar la normalización a un valor ya canónico devuelve ese mismo valor sin alterarlo. Como el catálogo se reimporta periódicamente, un dato ya normalizado puede reprocesarse sin corromperse (por ejemplo, sin que un valor ya en forma canónica se transforme de nuevo y deje de ser válido). Gracias a ello, la ingesta es segura de repetir, lo que simplifica la operación del sistema.

Persistencia idempotente y tolerante a fallos

La inserción en la base de datos se ha diseñado también con dos garantías. La primera es la idempotencia a nivel de fila: la sentencia de inserción utiliza la cláusula `ON CONFLICT` sobre la columna `url` de modo que, si una versión ya existe (identificada por su clave de negocio), sus datos se *actualizan* en lugar de provocar un error o un duplicado. Así, el catálogo converge siempre al estado de la última extracción, tanto si se trata de una carga inicial como de una actualización.

La segunda garantía es la **tolerancia a fallos parciales**. Cada versión se inserta dentro de su propio punto de guardado (*savepoint*) anidado, de forma que si una fila concreta provoca un error en la base de datos, solo se revierte esa fila y las demás se conservan. El proceso, además, no se limita a guardar: devuelve un informe detallado del resultado de la importación —cuántos registros se leyeron, cuántos se validaron correctamente, cuántos se normalizaron y cuántos se persistieron, junto con la lista de los que fallaron y por qué—. Este enfoque prioriza la *robustez* sobre la atomicidad estricta: dado que los datos provienen de un *scraping* de una web externa y, por su naturaleza, son imperfectos, es preferible cargar todo lo que sea correcto e informar de lo que no, antes que rechazar un lote completo por unas pocas filas defectuosas.

5.2. MODELO DE DATOS

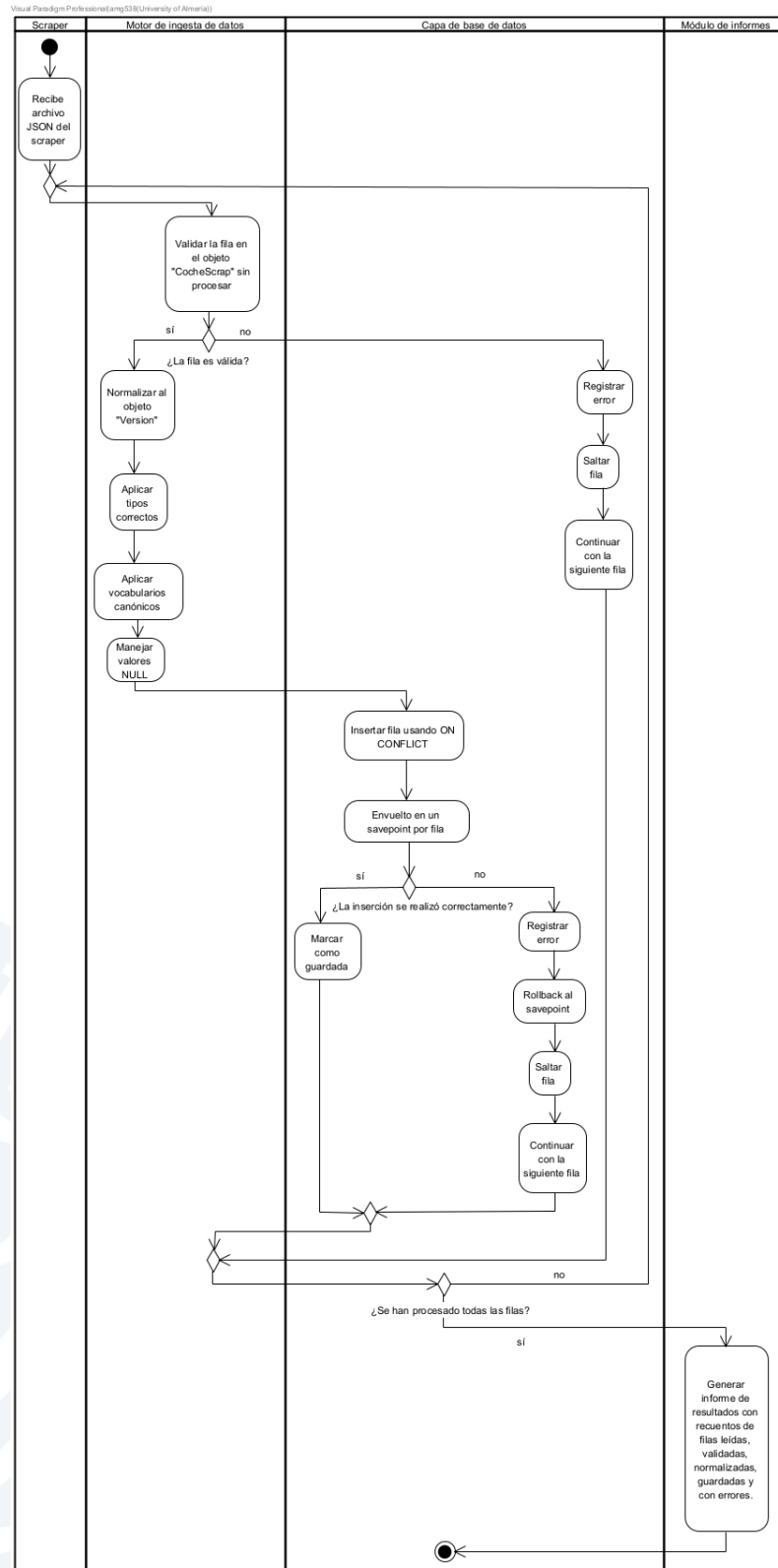


Figura 5.3: Diagrama de flujo del proceso de normalización e ingesta.

5.3 DISEÑO DEL *backend* (API REST)

El *backend* contiene la lógica del sistema y expone sus capacidades a través de una **API REST**. Su diseño persigue tres objetivos: ofrecer a la interfaz un contrato de datos tipado, mantener centralizado y seguro el acceso a la base de datos, y orquestar el pipeline de inteligencia artificial. La API se organiza en dos familias de *endpoints* —los del catálogo y comparador, de naturaleza determinista, y el del asistente conversacional, de naturaleza generativa—, más un par de *endpoints* de servicio. La tabla 5.3 resume el conjunto.

Método	Ruta	Función
GET	/api/cars	Lista de versiones filtrada y paginada (catálogo)
GET	/api/cars/{id}	Ficha completa de una versión
POST	/api/compare	Fichas de varias versiones para comparar
GET	/api/filters/meta	Valores distintos disponibles para los filtros
POST	/chat	Consulta en lenguaje natural al asistente
POST	/ingest	Ingesta de un archivo del <i>scraper</i> (interno)
GET	/health	Comprobación de estado del servicio

Tabla 5.3: *Endpoints* de la API REST del *backend*.

Endpoints del catálogo y comparador

Los *endpoints* del catálogo dan servicio a la navegación clásica de la interfaz. El de listado acepta como parámetros de consulta un conjunto de filtros (marca, modelo, combustible, carrocería, tracción, transmisión y rangos de precio, potencia y plazas) junto con la paginación, y devuelve la lista de versiones que los cumplen. El de ficha individual recupera un vehículo por su identificador, y el de comparación recibe una lista de identificadores y devuelve sus fichas completas. Un *endpoint* auxiliar, el de metadatos de filtros, devuelve los valores realmente presentes en la base de datos para cada atributo categórico, lo que permite a la interfaz poblar dinámicamente sus desplegables sin codificar los valores manualmente.

Las respuestas de estos *endpoints* se ajustan al modelo canónico *Version*. El esquema de la respuesta se declara de forma explícita, con tres consecuencias: la documentación interactiva de la API se genera automáticamente, el *framework* valida la salida, y la interfaz puede definir un tipo «espejo» que refleja esa estructura, de modo que el contrato de datos entre *frontend* y *backend* queda fijado y verificable en ambos extremos.

El repositorio y la construcción segura de consultas

Toda la interacción con la base de datos se concentra en un único módulo, el **repositorio de vehículos**, que actúa como frontera entre la lógica de negocio y el SQL. Este patrón evita que las sentencias SQL se dispersen por el código y permite construir las consultas de forma dinámica a partir de un diccionario de filtros: el repositorio traduce cada filtro presente en una condición de la cláusula *WHERE* y omite los ausentes, soportando tanto igualdades simples

como filtros de pertenencia a un conjunto (por ejemplo, varios tipos de combustible a la vez) y rangos numéricos.

El **tratamiento de la seguridad** de este módulo merece detalle. Todos los valores aportados por el usuario se pasan a la consulta como **parámetros vinculados**, nunca por interpolación de cadenas, lo que elimina por construcción el riesgo de inyección de SQL. En los pocos casos en que es inevitable insertar un identificador en la propia estructura de la consulta —por ejemplo, el nombre de la columna por la que ordenar o agregar—, ese identificador se valida previamente contra una **lista blanca** de valores permitidos: el nombre de ordenación se busca en un diccionario cerrado de órdenes válidos, y los campos y operaciones de agregación se comprueban contra conjuntos predefinidos. Cualquier valor fuera de esas listas se rechaza antes de construir la consulta. De este modo se concilian la flexibilidad de las consultas dinámicas con la imposibilidad de que un valor arbitrario llegue al motor de la base de datos.

Endpoints de servicio y configuración

Completan la API dos *endpoints* de servicio. El de comprobación de estado permite que la orquestación verifique que el *backend* está vivo antes de arrancar los servicios que dependen de él. El de ingesta es el que invoca el *scraper* para solicitar la carga de un archivo; por seguridad, su diseño restringe el acceso al directorio compartido y rechaza cualquier intento de acceder a rutas fuera de él. Toda la configuración sensible del *backend* (la cadena de conexión a la base de datos, la clave del proveedor de IA, los orígenes permitidos para las peticiones de la interfaz) se externaliza mediante variables de entorno y se valida al arranque, de forma que el código no contiene ningún secreto y el comportamiento puede ajustarse entre entornos sin recompilar. Adicionalmente, el *backend* incorpora sendos mecanismos opcionales de limitación de frecuencia de peticiones y de autenticación por clave para los *endpoints* sensibles, desactivados por defecto y activables por configuración.

5.4 DISEÑO DEL PIPELINE DE CONSULTA CON IA

El pipeline de inteligencia artificial es lo que materializa el objetivo principal de *autoAI*: permitir que un usuario exprese sus necesidades en **lenguaje natural** y reciba una recomendación **fundamentada en datos reales**. El problema de diseño consiste en conciliar la flexibilidad de un modelo de lenguaje, que puede interpretar peticiones ambiguas y redactar respuestas naturales, con el **rigor factual** de una base de datos, que es la única fuente legítima de cifras. La solución adoptada es no permitir que el modelo invente datos, sino obligarlo a obtenerlos de la base de datos a través de un conjunto acotado de herramientas, mediante el patrón conocido como *tool-calling*.

De lenguaje natural a filtros estructurados: las herramientas

En lugar de pedir al modelo que genere SQL —lo que sería potente pero arriesgado—, el diseño le ofrece un **catálogo cerrado de cuatro herramientas** de alto nivel que encapsulan las operaciones que el sistema sabe hacer con seguridad (tabla 5.4). La tarea del modelo se reduce entonces a traducir la intención del usuario en una llamada a la herramienta adecuada con los argumentos correctos; es decir, a convertir el lenguaje natural en un conjunto de **filtros estructurados**.

Herramienta	Función
buscar_coches	Busca versiones que cumplen unos filtros, con orden y límite opcionales
obtener_coche	Devuelve la ficha completa de un vehículo por su identificador
comparar	Devuelve las fichas de varios vehículos para compararlos
agregar	Calcula una agregación (recuento, media, mínimo, máximo) sobre un campo, opcionalmente agrupada; es la fuente de las gráficas

Tabla 5.4: Herramientas (*tools*) disponibles para el modelo de lenguaje.

El elemento determinante es la **especificación de los argumentos** de estas herramientas. El esquema de filtros que se entrega al modelo es un contrato estricto: enumera explícitamente los atributos por los que se puede filtrar y, para los categóricos, restringe los valores admisibles al vocabulario canónico definido en la normalización. Aquí se aprovecha la unificación de vocabularios: como el campo de combustible solo admite los nueve valores canónicos, el modelo no puede pedir un «híbrido» genérico ni un «diésel» con otra grafía, sino que debe traducir la intención del usuario a los valores exactos que existen en la base de datos. El propio esquema incluye, a modo de instrucciones, las equivalencias entre el lenguaje coloquial y el vocabulario canónico, lo que guía al modelo en esa traducción.

El bucle de *tool-calling* sobre datos reales

El componente central del pipeline es un **bucle de razonamiento** que alterna entre el modelo de lenguaje y la ejecución de herramientas. Funciona así: se envía al modelo la conversación junto con la descripción de las herramientas; si el modelo decide llamar a una o varias, el *backend* las ejecuta contra la base de datos y le devuelve los resultados; el modelo razona entonces sobre esos datos y decide si necesita más información (refinar la búsqueda, comparar, agregar) o si ya puede responder. El ciclo se repite hasta que el modelo deja de pedir herramientas, con un **límite máximo de iteraciones** que evita bucles infinitos y acota el coste de cada consulta.

Este diseño garantiza la invariante fundamental del sistema: **toda cifra que aparece en una respuesta procede de una consulta real a la base de datos**. El modelo aporta la interpretación, la estrategia de búsqueda y la redacción, pero los datos los pone siempre la base de datos. Sobre esta invariante se articula además un comportamiento experto, codificado en las instrucciones de sistema que acompañan a cada consulta: el modelo recibe directrices sobre cómo interpretar peticiones vagas («algo para la familia», «un coche para ciudad») traduciéndolas a filtros concretos, cómo priorizar por defecto el mercado actual, cómo actuar cuando el usuario filtra por un atributo que el sistema no soporta (explicarlo y ofrecer igualmente una búsqueda aproximada) y cómo tratar casos delicados como las unidades de consumo de los híbridos enchufables.

El diseño distingue entre **consultas objetivas** y **consultas exploratorias**. Cuando el usuario pide algo con un criterio inequívoco («el más barato», «el más potente»), la búsqueda devuelve ese resultado de forma determinista. Cuando la petición es cualitativa y sin un orden objetivo («un coche bonito para ciudad»), el sistema recupera un conjunto de candidatos más amplio y selecciona entre ellos con un muestreo sesgado hacia los más relevantes, de manera

que dos consultas parecidas no devuelvan siempre los mismos vehículos. Así se evita que el asistente resulte repetitivo sin mostrar por ello opciones de peor calidad.

Generación de respuestas estructuradas y explicadas

Una recomendación de coches no se expresa bien solo con texto: a menudo requiere una tabla comparativa, una gráfica de precios o la fotografía del vehículo. Por ello, la respuesta del asistente es un documento estructurado compuesto por una secuencia de **bloques tipados**. Se definen cuatro tipos de bloque: texto (en formato *Markdown*), tabla, gráfica (de barras o radar) e imagen. Una vez ha reunido los datos a través de las herramientas, el modelo emite una respuesta final que se ajusta de forma estricta a este esquema de bloques, lo que permite a la interfaz renderizar cada uno con el componente adecuado.

Este enfoque exige una **segunda fase** en el pipeline, separada del bucle de herramientas: una vez recopilados todos los datos, se realiza una última llamada al modelo solicitándole que produzca el documento de bloques en un formato verificable. El *backend* valida que la respuesta cumple el esquema y, si no lo cumple, reintenta una vez indicándole el error; si tampoco así se obtiene una respuesta válida, se entrega un bloque de texto de reserva en lugar de propagar un fallo a la interfaz.

Dos decisiones de diseño refuerzan en esta fase la integridad de los datos. La primera afecta a las **imágenes**: para evitar que el modelo invente o altere direcciones de fotografías, se le prohíbe emitir URLs; en su lugar, un bloque de imagen solo contiene el identificador del vehículo, y es el *backend* quien, en un paso posterior de «hidratación», recupera de la base de datos la URL real de la foto y la inserta en el bloque (descartándolo si el vehículo no existe o no tiene imagen). La segunda afecta a las **gráficas**: estas se alimentan exclusivamente de la salida de la herramienta de agregación, de modo que las barras de un gráfico reflejan siempre valores calculados por la base de datos y no estimaciones del modelo; las etiquetas, unidades y títulos son metadatos de presentación que nunca modifican las cifras subyacentes. En conjunto, estas medidas trasladan la invariante de «no inventar datos» también a los elementos visuales de la respuesta.

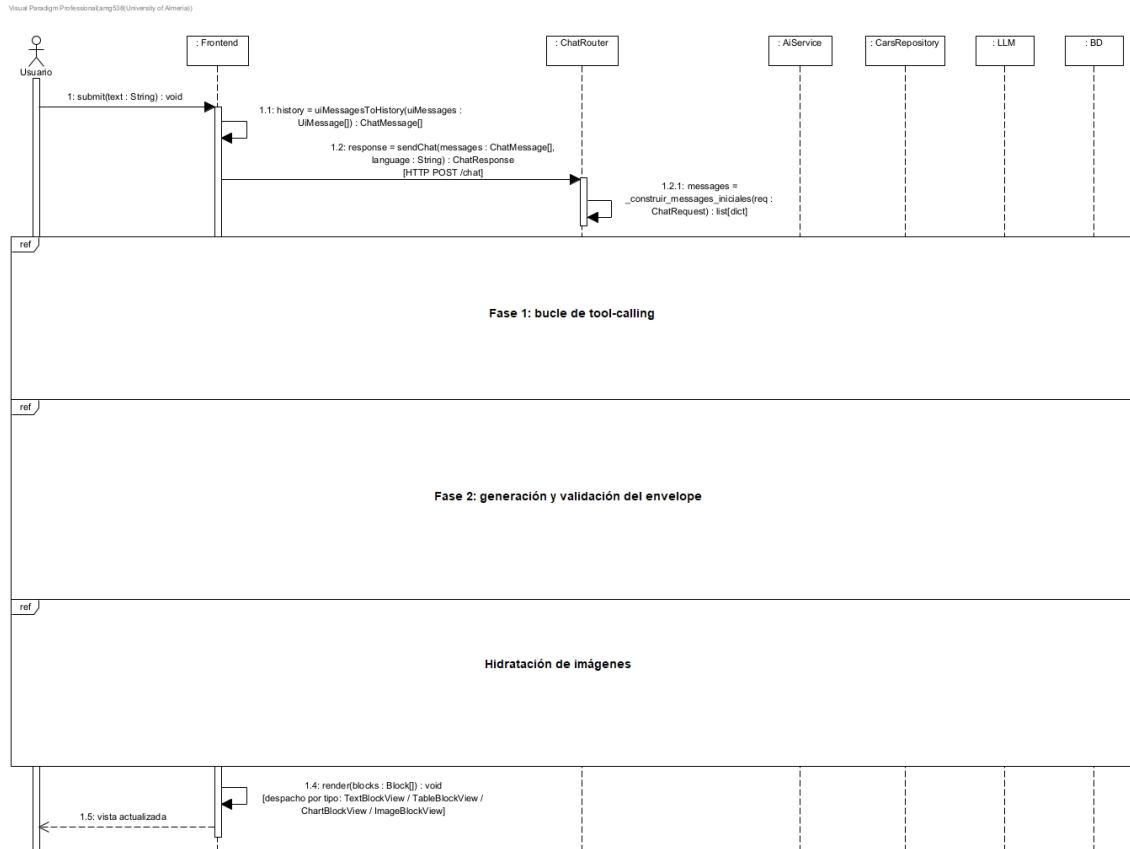


Figura 5.4: Diagrama de secuencia del pipeline de IA. Vista general.

5.4. DISEÑO DEL PIPELINE DE CONSULTA CON IA

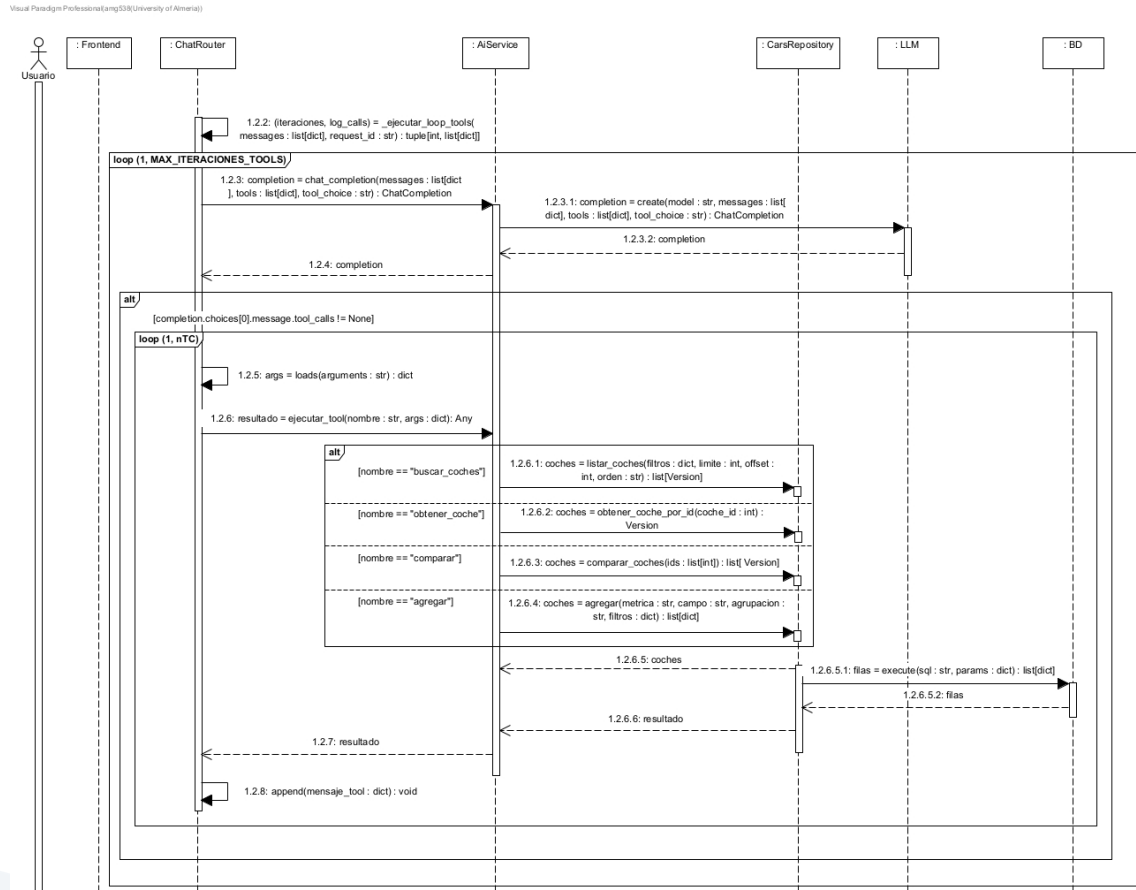


Figura 5.5: Diagrama de secuencia del pipeline de IA. Bucle de tool-calling.

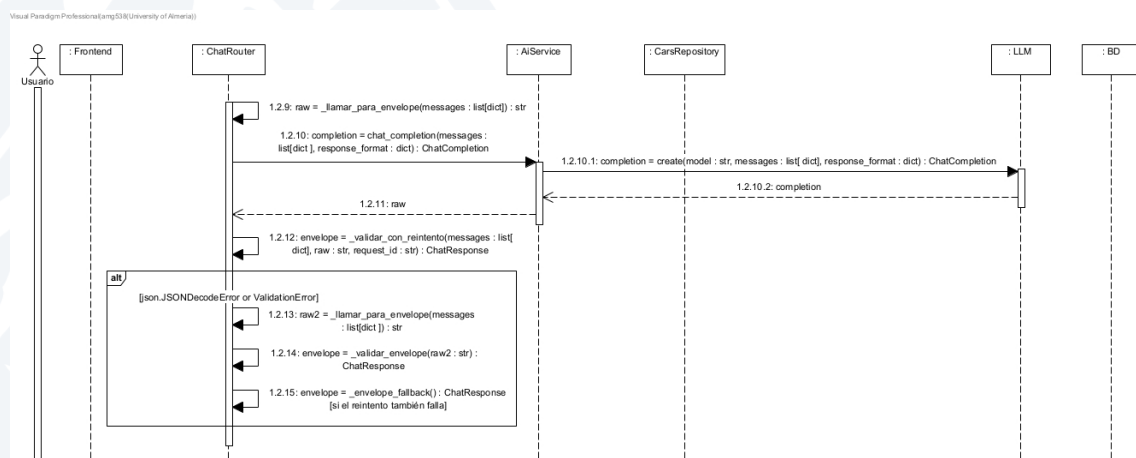


Figura 5.6: Diagrama de secuencia del pipeline de IA. Generación del envelope.

Visual Paradigm Professionals(amg538/University of Almería)

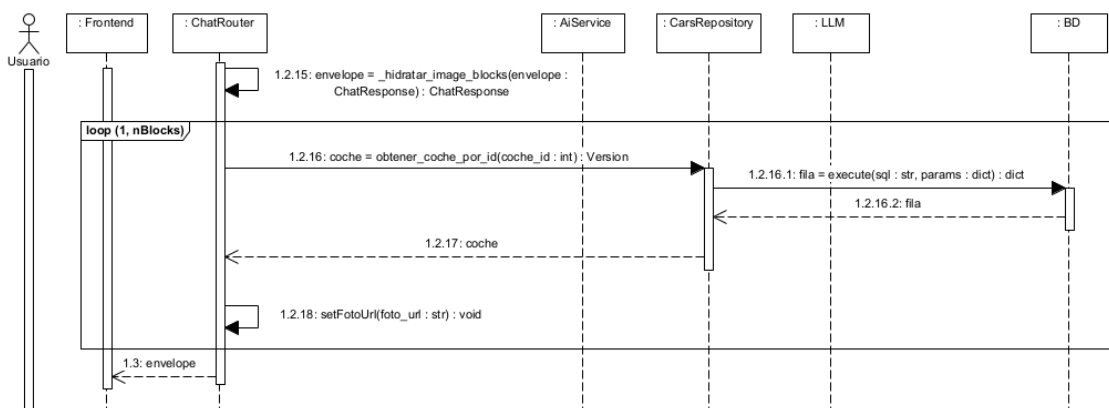


Figura 5.7: Diagrama de secuencia del pipeline de IA. Hidratación de imágenes.

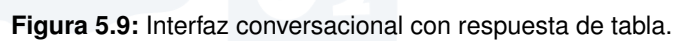
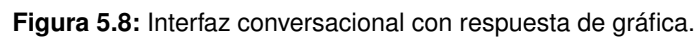
5.5 DISEÑO DE LA INTERFAZ DE USUARIO

La interfaz de usuario se ha diseñado como una **aplicación web de página única**, que actualiza la vista sin recargar la página, organizada en torno a una barra lateral de navegación que da acceso a sus cuatro áreas funcionales: el **asistente conversacional**, el **catálogo**, el **historial** de conversaciones y los **favoritos**. El diseño ofrece dos modos de exploración complementarios sobre el mismo catálogo: uno *conversacional*, guiado por la IA, para el usuario que prefiere expresar sus necesidades con sus propias palabras; y otro *estructurado*, mediante filtros explícitos, para quien prefiere acotar la búsqueda por su cuenta.

Interfaz conversacional

El asistente es la vista principal y se presenta inicialmente con una pantalla de bienvenida que invita a escribir la primera consulta. A partir de ahí, la interacción adopta la forma de una conversación, en la que se alternan los mensajes del usuario y las respuestas del asistente. El problema de diseño propio de esta vista es la **renderización de las respuestas estructuradas**: como la respuesta del *backend* es una secuencia de bloques tipados, la interfaz recorre esos bloques y delega cada uno en un componente especializado —el texto se interpreta como *Markdown*, las tablas se pintan con desplazamiento horizontal, las gráficas se dibujan con una biblioteca de visualización y las imágenes se muestran como fichas con una acción para añadir el vehículo a favoritos—. Este despacho por tipo de bloque permite que una misma respuesta combine, por ejemplo, un párrafo introductorio, una tabla comparativa y una gráfica de precios.

La vista atiende varios detalles de la interacción: mientras el *backend* procesa la consulta se muestra un indicador de escritura, la vista se desplaza automáticamente al último mensaje, los errores de comunicación se presentan como un mensaje del asistente en lugar de como un fallo abrupto, y junto al campo de entrada figura de forma permanente un aviso de que la IA puede cometer errores, en línea con un uso responsable de la herramienta.



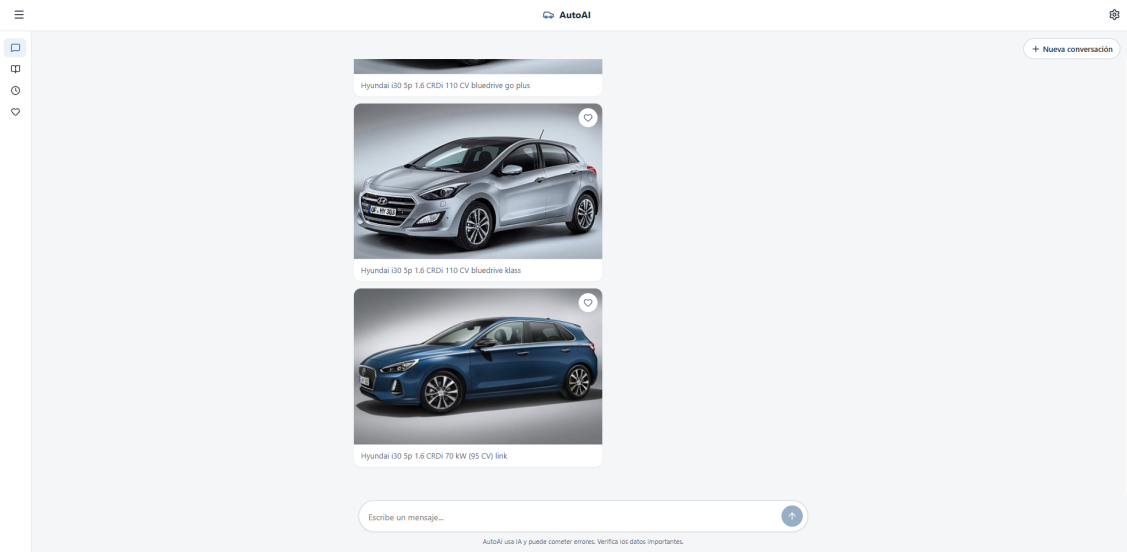


Figura 5.10: Interfaz conversacional con respuesta de imágenes.

Catálogo, filtros y comparador

El catálogo ofrece la vía de exploración estructurada. Presenta los vehículos en una rejilla de tarjetas y, sobre ella, un panel de filtros que se construye dinámicamente a partir de los metadatos que expone el *backend*: los desplegables de marca, combustible, carrocería, tracción y transmisión se pueblan con los valores realmente presentes en la base de datos, lo que evita ofrecer opciones sin resultados. Se complementan con campos para acotar rangos de precio, potencia y número de plazas, y con un buscador de modelo por texto. Los resultados se entregan paginados, y la vista contempla de forma explícita los estados de carga (mostrando una rejilla de marcadores de posición), de error (con opción de reintento) y de búsqueda sin resultados, de modo que la interfaz nunca queda en un estado ambiguo.

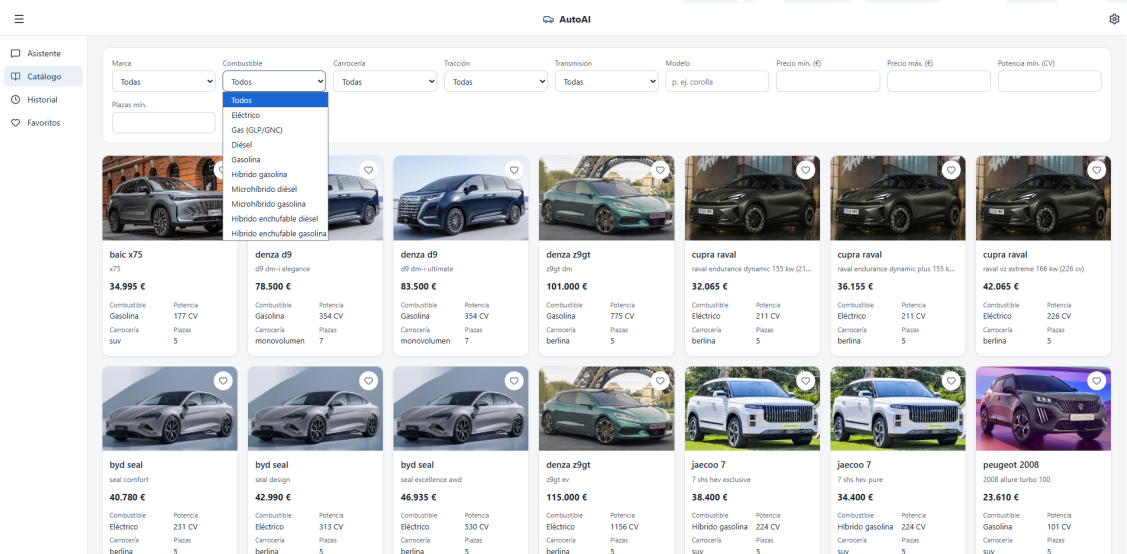


Figura 5.11: Vista de catálogo con filtros.

Favoritos e historial de consultas

Por último, el diseño incluye dos funcionalidades de personalización —**favoritos e historial**—, ambas resueltas en el lado del cliente: se persisten en el almacenamiento local del navegador, sin cuentas de usuario ni almacenamiento en el servidor. Esta opción es coherente con el alcance del proyecto, que no contempla la gestión de usuarios, y simplifica el *backend*, a costa de que la información quede ligada al navegador concreto.

Los favoritos permiten al usuario marcar vehículos de interés (desde las fichas con imagen que genera el asistente o desde el catálogo) y revisarlos después en una vista propia. El historial conserva las conversaciones mantenidas con el asistente, agrupadas en sesiones con un título derivado de la primera pregunta, de manera que el usuario pueda retomarlas o consultarlas más adelante. Para que estas vistas se mantengan siempre sincronizadas con los cambios —incluso entre varias pestañas abiertas simultáneamente—, la lectura del almacenamiento local se integra con el sistema de reactividad de la interfaz, de modo que cualquier modificación se refleja de inmediato en todas las vistas afectadas.

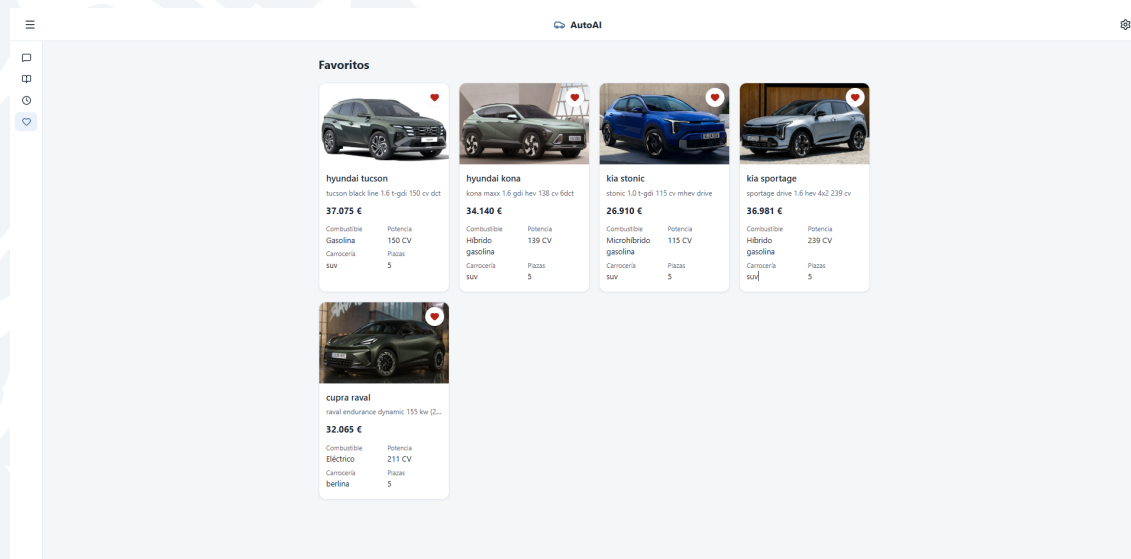


Figura 5.12: Vista de favoritos.

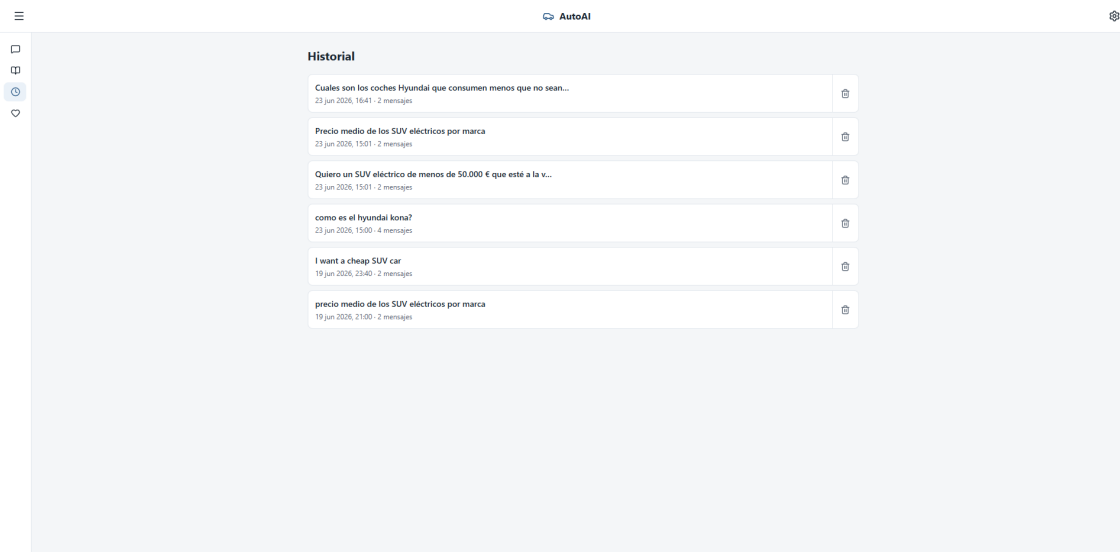


Figura 5.13: Vista de historial.

Preferencias de accesibilidad

La interfaz incorpora un panel de **preferencias de accesibilidad** que permite al usuario adaptar la presentación a sus necesidades. Se presenta como un cajón lateral (*drawer*) accesible —diseñado como diálogo modal con atrapado del foco, cierre con la tecla Esc y navegación completa por teclado— y reúne tres ajustes: el **tema** (claro u oscuro), el **tamaño de la fuente** (reducido, normal o ampliado) y el **idioma** de la interfaz (español o inglés). Coherentemente con la decisión adoptada para favoritos e historial, estas preferencias también se resuelven en el lado del cliente y persisten en el almacenamiento local del navegador, con unos valores por defecto definidos (tema claro, tamaño normal e idioma español) y una lectura defensiva que valida cada campo por separado y recurre al valor por defecto ante cualquier dato corrupto.

El diseño de este panel persigue que cada ajuste tenga un efecto global e inmediato sobre toda la aplicación, para lo cual cada preferencia se traduce en un cambio sobre el documento raíz. El tema alterna un atributo en la raíz del documento que conmuta la paleta de colores de toda la interfaz. El tamaño de fuente, dado que toda la tipografía está expresada en unidades relativas, se controla escalando un único valor en la raíz, lo que redimensiona la interfaz *proporcionalmente* sin romper la maquetación. El idioma, por su parte, ajusta tanto el atributo de idioma del documento como el sistema de internacionalización que traduce los textos de la interfaz; conviene recordar aquí que la localización solo afecta a los textos propios de la aplicación y a la prosa generada por el asistente, nunca a los datos de los vehículos.

Una última consideración de diseño atañe a la **experiencia de arranque**: para evitar el destello visual que se produciría si la aplicación se cargara primero con el tema por defecto y cambiara al tema preferido una vez iniciada, las preferencias guardadas se aplican al documento *antes* de que se monte la interfaz. Así, el usuario percibe desde el primer instante la presentación que había elegido.

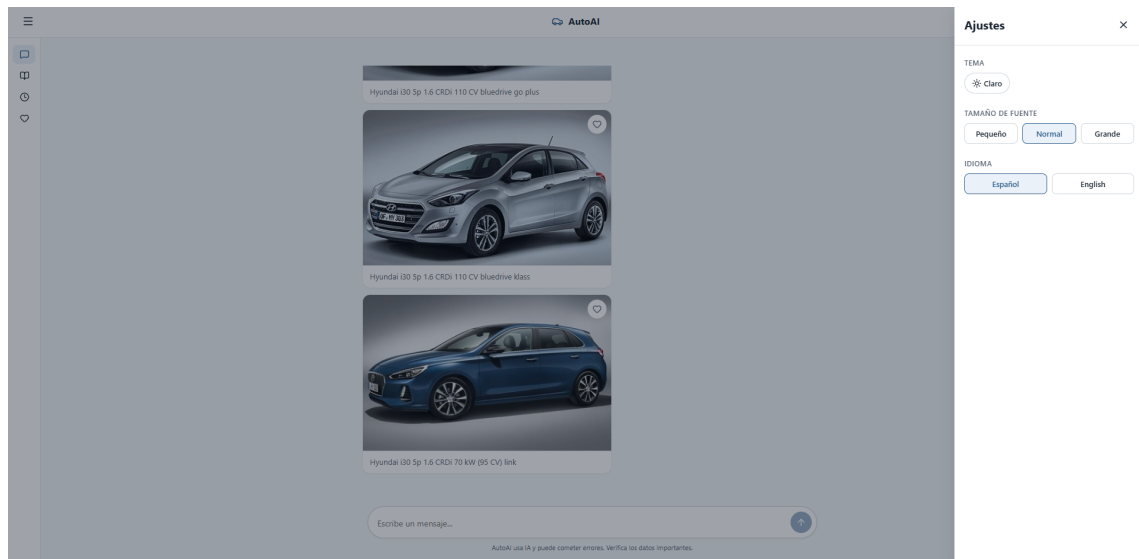


Figura 5.14: Vista del panel de preferencias de accesibilidad.

6 IMPLEMENTACIÓN

Tras el diseño del capítulo anterior, este capítulo expone la implementación: las tecnologías empleadas, las decisiones de codificación relevantes y los fragmentos de código que sustentan cada componente. La estructura del capítulo sigue el recorrido del dato a través del sistema, desde su extracción del portal de origen hasta su presentación en la interfaz, pasando por la base de datos, el *backend*, el pipeline de inteligencia artificial y la contenerización. En cada apartado se justifican las elecciones técnicas y se señalan los puntos del código donde se resuelven los problemas ya anticipados en el diseño.

6.1 OBTENCIÓN DE DATOS: SCRAPING DE KM77.COM

La obtención de datos se ha implementado con *Scrapy*, un *framework* de *scraping* asíncrono para Python. Se eligió por varias razones. Maneja de forma nativa la concurrencia y el encolado de peticiones, e incluye mecanismos de control de carga (*auto-throttle*, retardos entre peticiones) necesarios para no saturar el servidor de origen. Además, permite expresar la navegación por el sitio como una cascada de *callbacks*, que encaja con la estructura jerárquica del portal. Por último, exporta el resultado directamente a JSON, el formato que después consume el *backend*.

6.1.1 Diseño del *spider* con Scrapy

El portal `km77.com` organiza su catálogo de forma jerárquica, y el *spider* (definido en `km77_spider.py`) lo recorre reproduciendo esa jerarquía en cinco niveles encadenados, cada uno con su propio *callback* que extrae enlaces y los sigue hacia el nivel siguiente:

1. **Marcas** (`parse`): a partir de la página índice de coches se extraen los enlaces de cada marca.
2. **Modelos** (`parse_brand`): de la página de cada marca se obtienen sus modelos.
3. **Submodelos** (`parse_model`): de cada modelo se recogen sus generaciones o submodelos. En este nivel se captura la URL de la fotografía (`foto_url`), que solo está disponible aquí.
4. **Versiones** (`parse_submodel`): de cada submodelo se listan sus versiones o acabados, junto con la fecha de comercialización y la capacidad de maletero, que figuran en la tabla del submodelo.

5. **Ficha técnica** (*parse_version*): finalmente, de la página de cada versión se extraen todos los datos técnicos (precio, potencia, dimensiones, consumo, etc.) y se emite el registro final.

La información parcial que se necesita en los niveles inferiores (nombre de la marca, fotografía, fecha y maletero) se traslada entre *callbacks* mediante el diccionario *meta* de las peticiones de Scrapy, de modo que el registro final reúne datos procedentes de varios niveles de la jerarquía.

Las páginas de ficha técnica presentan los datos en tablas cuya estructura no es homogénea: unas etiquetas están en celdas de encabezado (<th>) y otras en celdas de datos (<td>). Para localizar cada dato con independencia de esas variaciones, la extracción combina selectores CSS con expresiones *XPath* que buscan una fila por el texto de su etiqueta. Se definieron dos funciones auxiliares (*xpath_row* y *xpath_cell_all*) que encapsulan ambos casos, lo que hace la extracción robusta frente a la disposición concreta de cada tabla.

Por decisión de diseño, el *spider* no transforma ni valida los datos: extrae cada campo como texto crudo, tal cual aparece en la página (con unidades, símbolos de moneda y marcadores de dato ausente), y lo vuelca sin más procesamiento. El saneamiento se delega íntegramente en el *backend*, lo que mantiene separadas la producción de datos crudos y su validación y persistencia, tal como se justificó en el diseño. El resultado se exporta a *km77_output.json* mediante la configuración de *feeds* de Scrapy.

El listado 6.1 muestra esta cascada: cada *callback* extrae los enlaces de su nivel y los sigue (*response.follow*) hacia el siguiente, trasladando la información parcial en el diccionario *meta*.

```

1 class Km77Spider(scrapy.Spider):
2     name = "km77_spider"
3     start_urls = ["https://www.km77.com/coches"]
4
5     # NIVEL 1 – Marcas
6     def parse(self, response):
7         for brand_link in response.css("a.js-brand-item-link"):
8             brand_name = brand_link.attrib.get("name")
9             yield response.follow(
10                 brand_url,
11                 callback=self.parse_brand,
12                 meta={"brand_name": brand_name},
13             )
14
15     # NIVEL 2 – Modelos por marca
16     def parse_brand(self, response):
17         brand_name = response.meta["brand_name"]
18         for model_tag in response.css("a.d-block"):
19             yield response.follow(
20                 model_url,
21                 callback=self.parse_model,
22                 meta={"brand_name": brand_name, "model_name": model_name},
23             )
24
25     # NIVELES 3 y 4: parse_model -> parse_submodel (analogos)
26     # ...
27
28     # NIVEL 5 – Ficha tecnica de cada version
29     def parse_version(self, response):
30         # xpath_row localiza una celda por el texto de su etiqueta

```


6.1. OBTENCIÓN DE DATOS: SCRAPING DE KM77.COM

```
31 potencia = xpath_row("Potencia maxima")
32 # ... resto de campos como texto crudo, sin normalizar ...
33 yield {
34     "marca": response.meta.get("brand_name"),
35     "potencia": potencia,
36     # ...
37 }
```

Listado 6.1: Cascada de *callbacks* del *spider* (extracto de `km77_spider.py`).

6.1.2 Control de carga y respeto al portal de origen

Como el *scraping* accede a un servicio de terceros, su configuración se ha orientado a no causar perjuicio al portal. El *spider* declara un *user-agent* de navegador realista y un idioma de aceptación coherentes con una visita normal, y aplica varios límites de carga: un retardo entre peticiones (`DOWNLOAD_DELAY`), un tope de peticiones concurrentes y la extensión *auto-throttle* de Scrapy, que ajusta la cadencia según la latencia de respuesta del servidor y la reduce si el portal empieza a tardar más. El proyecto respeta además el fichero `robots.txt` del sitio (`ROBOTSTXT_OBEY`). Con ello se obtienen los datos sin degradar el servicio para otros usuarios.

El *spider* habilita asimismo la persistencia del estado de rastreo (`JOBDIR`), de manera que un *crawl* interrumpido puede reanudarse sin repetir las peticiones ya realizadas, lo que ahorra carga tanto al portal como al propio sistema.

6.1.3 Automatización periódica del scraping

El catálogo de un portal de coches cambia con el tiempo: aparecen modelos nuevos, se descatalogan versiones y se actualizan precios. Para que los datos de *autoAI* no queden obsoletos, el *scraping* se ejecuta de forma periódica y desatendida. La automatización se resuelve dentro del propio contenedor del *scraper* mediante *supercronic*, un planificador tipo *cron* pensado para contenedores (registra su salida por la salida estándar y no requiere un demonio del sistema).

El *script* de arranque del contenedor (`entrypoint.sh`) genera al vuelo una tabla *cron* a partir de la variable de entorno `SCRAPER_CRON`, cuyo valor por defecto es una ejecución semanal, y lanza *supercronic* sobre ella. Así se ajusta la frecuencia del *scraping* por configuración, sin tocar el código ni reconstruir la imagen.

Cada ejecución dispara el *script* `run_scrape.sh`, que encadena las dos fases del proceso: primero lanza el *crawl* de Scrapy y vuelca el resultado en el volumen compartido; después notifica al *backend* mediante una petición HTTP `POST` a su *endpoint* de ingesta con el nombre del archivo recién generado. Conforme al diseño, el *scraper* no inserta nada en la base de datos: solo produce el archivo y delega su ingesta. La llamada usa la opción `-fail-with-body` de `curl`, de modo que un fallo en la ingesta se refleja en el código de salida y queda registrado en los *logs* del contenedor.

El listado 6.2 recoge el *script* `run_scrape.sh`.

```
1 #!/bin/sh
2 set -e
3
```

```
4 OUTPUT="${SCRAPER_OUTPUT:-/data/km77_output.json}"
5 BACKEND="${BACKEND_URL:-http://backend:8000}"
6
7 # /ingest interpreta 'file' relativo a INGEST_DIR (/data): enviamos solo el nombre
8 OUTPUT_NAME="$(basename "${OUTPUT}")"
9
10 # Fase 1: scraping -> volumen compartido
11 cd /app
12 scrapy crawl km77_spider -O "${OUTPUT}"
13
14 # Fase 2: notificar al backend para que ingeste el archivo
15 curl -sS --fail --with-body -X POST -H "Content-Type: application/json" \
16   --data "{\"file\":\"${OUTPUT_NAME}\"}" \
17   "${BACKEND}/ingest"
```

Listado 6.2: Encadenado de *crawl* e ingesta (*run_scrape.sh*).

6.2 PROCESO ETL Y CARGA EN LA BASE DE DATOS

El proceso de extracción, transformación y carga (ETL) reside íntegramente en el *backend*, concentrado en el módulo *services/normalization.py*, y se activa cuando el *scraper* invoca el *endpoint* */ingest*. La función orquestadora, *ingest_desde_archivo*, encadena cuatro etapas (lectura, validación, normalización y persistencia) y devuelve un informe detallado de cada una. Las tres últimas se ejecutan de forma independiente para cada registro, lo que permite aislar los fallos individuales sin abortar el lote completo, conforme a una de las garantías del diseño.

6.2.1 Validación y modelos de datos

El sistema utiliza *Pydantic* para definir y validar todas sus estructuras de datos. El ciclo de vida del dato se modela con dos clases, que corresponden a los dos momentos descritos en el diseño:

- *CocheScrap* (en *models/scraped.py*) es la representación cruda: todos sus campos son de tipo *str*, porque así llegan del *scraping*. Al construir un *CocheScrap* a partir de cada elemento del JSON, *Pydantic* verifica que están todos los campos esperados; un registro malformado lanza una excepción que se captura y se registra como error de validación, sin interrumpir el resto.
- *Version* (en *models/normalized.py*) es el modelo canónico del sistema, con cada atributo ya tipado (*int*, *float*, *datetime.date*, *str*) y con los campos que pueden faltar declarados como opcionales (*| None*). Es la estructura común a todas las capas: la que produce la normalización, la que devuelve la base de datos y la que se serializa hacia la interfaz.

6.2.2 Saneamiento y unificación de vocabularios

La transformación de *CocheScrap* a *Version* la realiza la función *normalizar_coche*, apoyada en funciones de parseo especializadas por tipo de campo. La parte mecánica corresponde a los parsers escalares, que extraen de cada cadena el valor numérico o la fecha

que contiene y descartan lo accesorio: `parse_potencia` conserva la cifra en caballos de «177 CV / 130 kW»; `parse_dimension_mm` elimina el separador de miles de «4.165 mm»; `parse_consumo` retira las unidades de «7,2 l/100 km» y normaliza la coma decimal; y `parse_fechas` descompone un rango «(03/2021 - 09/2023)» en sus dos fechas. Todos ellos se construyen sobre una función común, `parse_opcional`, que aplica el principio transversal del diseño: ante un valor ausente o el marcador de dato faltante del portal (un guion), la conversión no falla ni inventa, sino que devuelve `None`.

Más delicada es la unificación de los vocabularios categóricos, y en particular la del combustible. Como el portal no clasifica los vehículos por su grado de electrificación, sino que deja esa información implícita en el nombre comercial, la función `normalizar_combustible` la deriva en dos pasos: primero determina la base fósil a partir del campo de origen (`_base_fosil`) y después analiza el texto del nombre y el submodelo con tres expresiones regulares (`_RX_PHEV`, `_RX_HEV`, `_RX_MHEV`) que reconocen los términos comerciales. Estas se aplican en orden de mayor a menor especificidad (enchufable antes que autorrecargable, y este antes que microhíbrido) para evitar clasificaciones erróneas, y el resultado es uno de los nueve valores del vocabulario canónico. La normalización de la carrocería (`normalizar_carroceria`) sigue una estrategia análoga, con una tabla de correspondencias exactas y reglas por subcadena que absorben sinónimos.

Estas funciones son idempotentes: al detectar la base fósil reconocen tanto los valores crudos del portal como los ya canónicos, de modo que reprocesar un dato ya normalizado devuelve el mismo valor. Gracias a ello, la reimportación periódica del catálogo es segura de repetir.

La tabla 6.1 recoge varios ejemplos del resultado de este proceso, contrastando el valor crudo extraído del portal con el valor ya normalizado y tipado que se persiste en la base de datos.

Campo	Entrada cruda (texto)	Salida normalizada
potencia	"177 CV / 130 kW"	177.0 (float)
longitud	"4.165 mm"	4165.0 (float)
consumo_medio	"7,2 l/100 km"	7.2 (float)
precio	"35.900 €"	35900.0 (float)
fecha_inicio	"(03/2021 - 09/2023)"	2021-03-01,
fecha_fin		2023-09-01
capacidad_maletero	"_"	NULL
numero_marchas	"Múltiples" (e-CVT)	NULL
combustible	"Gasolina" + nombre con «e-Hybrid»	phev_gasolina
combustible	consumo en "kWh/100 km" (unidad eléctrica)	electrico
carroceria	"SUV/Todoterreno"	suv

Tabla 6.1: Ejemplos de saneamiento y unificación de vocabularios: del texto crudo del portal al valor canónico tipado.

6.2.3 Persistencia idempotente y tolerante a fallos

La carga en la base de datos (`guardar_en_bd`) se implementa con *psycopg* (versión 3) y aplica las dos garantías del diseño. La idempotencia a nivel de fila se consigue con una sentencia `INSERT ... ON CONFLICT (url) DO UPDATE`: si una versión ya existe, identificada por su clave de negocio (la `url`), sus datos se actualizan en lugar de provocar un duplicado, de manera que el catálogo converge al estado de la última extracción. Todos los valores se pasan a la consulta como parámetros con nombre (`%(campo)s`), nunca por interpolación de cadenas.

La tolerancia a fallos parciales se logra con un *savepoint* anidado por cada inserción, mediante el gestor de contexto `conn.transaction()` de *psycopg* dentro del bucle de filas. Si una fila concreta provoca un error en la base de datos, solo se revierte su *savepoint*; las filas correctas previas se conservan y el contador de insertados permanece exacto. El error se acumula en una lista de fallos que forma parte del informe final. Este informe, devuelto por `ingest_desde_archivo` y propagado como respuesta del *endpoint* `/ingest`, recoge cuántos registros se leyeron, validaron, normalizaron y guardaron, junto con la lista de los que fallaron y su motivo. Se prioriza así la robustez sobre la atomicidad estricta: ante datos imperfectos procedentes de una web externa, es preferible cargar todo lo correcto e informar de lo demás que rechazar el lote entero.

6.2.4 Esquema de la base de datos

La base de datos es una instancia de *PostgreSQL* cuyo esquema se define en `bd/init.sql`, un *script* que el contenedor de *PostgreSQL* ejecuta automáticamente en su primera inicialización. Implementa la única tabla `cars`, desnormalizada según se justificó en el diseño, con sus veintiséis columnas, la clave primaria `id`, la restricción `UNIQUE` sobre `url` y el conjunto de índices sobre las columnas de filtrado más habituales. Entre estos destaca el **índice parcial** `idx_cars_vigentes`, restringido a las filas con `fecha_fin IS NULL`, que acelera la consulta «solo vehículos a la venta» ocupando una fracción del espacio de un índice completo. Los índices que dan soporte al orden por defecto y al filtro de vigencia se recogen además en una migración idempotente (`001_indices_fecha.sql`), aplicable sobre una base ya poblada sin recrear el volumen de datos.

El acceso desde el *backend* se gestiona con un **pool de conexiones** (`psycopg_pool`), configurado en `core/db.py`, que se abre al arrancar la aplicación y se reutiliza entre peticiones. Las conexiones se configuran para devolver las filas como diccionarios (`dict_row`), lo que permite construir directamente los modelos *Version* a partir de cada fila.

6.3 BACKEND CON FASTAPI

El *backend* se ha implementado con *FastAPI*, un *framework* web de Python que aprovecha las anotaciones de tipos y se integra de forma nativa con *Pydantic*. Esa integración motivó su elección: al declarar los modelos de entrada y salida con tipos, *FastAPI* valida las peticiones, serializa las respuestas y genera documentación interactiva de la API sin código adicional. El servidor se ejecuta sobre *uvicorn* (servidor ASGI) y, en producción, con varios procesos de trabajo.

La aplicación se compone en `main.py`, que crea la instancia de FastAPI, registra los enrutadores de las familias de *endpoints*, configura el *middleware* de CORS a partir de los orígenes permitidos por configuración y gestiona el ciclo de vida del *pool* de conexiones mediante un *lifespan*, que lo abre al arrancar y lo cierra al terminar. La estructura del código respeta la arquitectura en capas del diseño: la capa de API (paquete `api`), la de servicios (`services`), la de modelos (`models`) y el núcleo transversal (`core`).

6.3.1 Endpoints del catálogo y comparador

Los *endpoints* del catálogo se definen en `api/cars.py`. El de listado (`GET /api/cars`) declara cada filtro como un parámetro de consulta con su tipo y sus restricciones (por ejemplo, los rangos de precio y potencia se validan como no negativos, y la paginación se acota a un máximo por página), recoge los filtros en un diccionario y delega en el repositorio. Se completan con el *endpoint* de ficha individual (`GET /api/cars/{id}`), que responde con un error 404 si el vehículo no existe; el de comparación (`POST /api/compare`), que recibe una lista de identificadores; y el de metadatos de filtros (`GET /api/filters/meta`), que devuelve los valores realmente presentes en la base de datos para poblar dinámicamente los desplegables de la interfaz. Todas estas respuestas se declaran con `response_model`, de modo que FastAPI valida la salida contra el modelo `Version` y fija el contrato de datos con el *frontend*.

6.3.2 El repositorio y la construcción segura de consultas

Toda la interacción con la base de datos se concentra en `services/cars_repository.py`, que actúa como frontera entre la lógica de negocio y el SQL. Las consultas del catálogo se construyen dinámicamente: la función `_construir_where` traduce cada filtro presente en el diccionario en una condición de la cláusula `WHERE` y omite los ausentes. Admite igualdades simples, rangos numéricos y filtros de pertenencia a un conjunto, que generan una condición `= ANY(...)` cuando llegan varios valores (por ejemplo, varios combustibles a la vez).

La seguridad es el aspecto más cuidado de este módulo. Los valores aportados por el usuario se pasan siempre como parámetros vinculados (`%(...)s`), nunca por interpolación de cadenas, lo que elimina por construcción el riesgo de inyección de SQL. En los pocos casos en que es inevitable insertar un identificador en la estructura de la consulta (el campo por el que ordenar, o el campo y la operación de una agregación), ese identificador se valida previamente contra una lista blanca: el orden se resuelve buscando la clave en un diccionario cerrado (`ORDENES_VALIDOS`), y la función `agregar` comprueba la métrica, el campo y la agrupación contra conjuntos predefinidos (`METRICAS_AGREGAR`, `CAMPOS_AGREGABLES`, `AGRUPACIONES_VALIDAS`) y lanza una excepción ante cualquier valor fuera de ellos antes de construir la consulta. Esta invariante de seguridad está documentada en el propio código para que no se relaje al añadir nuevos campos. Por último, todos los órdenes colocan los valores nulos al final (`NULLS LAST`) y añaden un desempate por `id`, lo que garantiza una paginación estable.

6.3.3 Endpoints de servicio y configuración

Completan la API el *endpoint* de comprobación de estado (`GET /health`), que la orquestación usa para verificar que el servicio está vivo, y el de ingesta (`POST /ingest`). Este último,

definido en `api/routes.py`, interpreta el nombre de archivo recibido como relativo al directorio compartido y, antes de abrirlo, comprueba que la ruta resuelta sigue estando dentro de ese directorio, rechazando con un error cualquier intento de salir de él (*path traversal*).

La configuración se centraliza en `core/config.py` mediante *pydantic-settings*: la cadena de conexión a la base de datos, la clave del proveedor de IA, el modelo a utilizar y los orígenes CORS se leen de variables de entorno y se validan al arranque, de forma que el código no contiene ningún secreto. Por último, el *backend* incorpora dos mecanismos de seguridad opcionales para los *endpoints* sensibles (`/chat` e `/ingest`), implementados como dependencias de FastAPI en `api/dependencies.py`: una limitación de frecuencia de peticiones por IP y una autenticación por clave de API. Ambos están **inertes por defecto** y solo se activan al definir las variables de entorno correspondientes; la comparación de la clave se hace en tiempo constante (`secrets.compare_digest`) para no filtrar información por el tiempo de respuesta.

6.4 INTEGRACIÓN DE LA INTELIGENCIA ARTIFICIAL

La integración con el modelo de lenguaje se realiza a través de la API de *chat completions* compatible con OpenAI, empleando su biblioteca cliente oficial para Python. La lógica se reparte en dos módulos: `api/chat.py`, que contiene el *endpoint* `/chat` y orquesta el pipeline completo, y `services/ai_service.py`, que encapsula el cliente del modelo, la definición de las herramientas y su ejecución contra el repositorio. Esta separación mantiene la lógica de IA aislada del resto del *backend* y facilita su prueba.

6.4.1 Traducción de lenguaje natural a filtros estructurados

El enfoque se basa en el patrón de *tool-calling*: en vez de pedir al modelo que genere SQL, se le ofrece un catálogo cerrado de cuatro herramientas de alto nivel (`buscar_coches`, `obtener_coche`, `comparar` y `agregar`), declaradas en la lista `TOOLS` de `ai_service.py`. Cada herramienta se describe con un esquema JSON que enumera sus argumentos, y la tarea del modelo se reduce a traducir la intención del usuario en una llamada a la herramienta adecuada con los argumentos correctos.

La pieza determinante es el esquema de filtros (`_FILTROS_SCHEMA`), que actúa como contrato estricto: para los atributos categóricos, restringe los valores admisibles mediante listas `enum` al vocabulario canónico definido en la normalización. Así, el campo de combustible solo acepta los nueve valores canónicos, lo que impide al modelo pedir un «híbrido» genérico o un «diésel» con otra grafía y le obliga a traducir la intención del usuario a los valores que existen en la base de datos. El propio esquema y las listas de `enum` se construyen a partir de las mismas constantes que valida el repositorio (`ORDENES_VALIDOS`, `CAMPOS_AGREGABLES...`), de modo que la descripción que recibe el modelo y la validación del servidor no pueden divergir.

Esta orientación experta se refuerza con un extenso *prompt* de sistema (`SYSTEM_PROMPT`) que codifica el conocimiento de dominio: las equivalencias entre el lenguaje coloquial y el vocabulario canónico, las reglas para interpretar peticiones vagas («algo para la familia», «un coche para ciudad»), la prioridad por defecto del mercado actual, el tratamiento de casos



delicados como el consumo de los híbridos enchufables y el procedimiento a seguir cuando el usuario filtra por un atributo no soportado. A este *prompt* se concatena una instrucción de idioma que afecta únicamente a la prosa generada, nunca a los datos del vehículo.

El listado 6.3 muestra un extracto del esquema de filtros: el atributo `combustible` se declara como un *array* cuyos elementos están restringidos por una lista *enum* al vocabulario canónico, de modo que el modelo no puede emitir un valor fuera de él.

```
1 _FILTROS_SCHEMA = {
2     "type": "object",
3     "properties": {
4         "combustible": {
5             "type": "array",
6             "items": {
7                 "type": "string",
8                 "enum": [
9                     "gasolina", "gasoleo", "gas", "electrico",
10                    "mhev_gasolina", "mhev_gasoleo", "hev_gasolina",
11                    "phev_gasolina", "phev_gasoleo",
12                ],
13            },
14            "description": (
15                "Fuente fosil + electrificacion. NO existe 'hibrido' ni "
16                "'diesel'. Mapea: 'diesel'->['gasoleo'], 'GLP'->['gas']..."
17            ),
18        },
19        # ... resto de filtros (carroceria, precio_min, solo_vigentes, ...) ...
20    },
21    "additionalProperties": False,
22 }
```

Listado 6.3: Extracto de `_FILTROS_SCHEMA`: el *enum* cerrado del combustible (`ai_service.py`).

6.4.2 Bucle de *tool-calling* sobre datos reales

El bucle de razonamiento se implementa en la función `_ejecutar_loop_tools` de `chat.py`. En cada iteración se envía al modelo la conversación junto con la descripción de las herramientas; si el modelo decide llamar a una o varias, el *backend* las ejecuta (`ai_service.ejecutar_tool`) contra la base de datos, añade los resultados a la conversación como mensajes de rol `tool` y reitera. El ciclo continúa hasta que el modelo deja de pedir herramientas, con un límite máximo de iteraciones (`MAX_ITERACIONES_TOOLS`) que acota el coste y evita bucles infinitos; un umbral de advertencia inferior registra en los *logs* las consultas anormalmente largas para su análisis. La ejecución de cada herramienta se envuelve en un `try/except`, de modo que un fallo concreto no aborta la conversación, sino que se devuelve al modelo como un resultado de error sobre el que puede razonar.

Este diseño garantiza la invariante del sistema: toda cifra que aparece en una respuesta procede de una consulta real a la base de datos. El modelo aporta la interpretación y la redacción, pero los datos los pone siempre la base de datos.

Dentro de `buscar_coche` se distingue entre consultas objetivas y exploratorias. Cuando la petición tiene un criterio inequívoco y el modelo solicita un orden explícito («el más barato», «el más potente»), la búsqueda devuelve ese resultado de forma determinista. Cuando la petición es cualitativa y se pide sin orden, se recupera un conjunto de candidatos más amplio y se selecciona entre ellos con un muestreo sesgado (`_submuestra_sesgada`): se asigna

a cada candidato un peso decreciente según su relevancia y se extraen sin reemplazo, de modo que los más relevantes salen con más probabilidad pero dos consultas parecidas no devuelven siempre los mismos vehículos. Así se evita que el asistente resulte repetitivo sin mostrar por ello opciones de peor calidad. La función contempla además que el modelo a veces aplane los argumentos (`_extraer_filtros`) y reconstruye entonces el diccionario de filtros de forma tolerante.

6.4.3 Generación de respuestas explicadas

Una vez recopilados los datos, una segunda fase del pipeline solicita al modelo la respuesta final como un documento estructurado de bloques tipados. Estos se definen en `models/chat.py` como una unión discriminada de Pydantic (Block) con cuatro variantes (texto, gráfica, tabla e imagen) y se exigen al modelo mediante un *response format* de tipo `json_schema` derivado del propio modelo (`_response_format`), lo que fuerza una salida verificable.

La validación de la respuesta es defensiva (`_validar_con_reintento`): el *backend* comprueba que el JSON cumple el esquema y, si no lo cumple, reintenta una vez devolviendo al modelo el mensaje de error concreto; si tampoco así se obtiene una respuesta válida, entrega un bloque de texto de reserva (`_envelope_fallback`) en lugar de propagar un fallo a la interfaz.

Dos decisiones de implementación trasladan la integridad de los datos a los elementos visuales. La primera es la hidratación de imágenes (`_hidratar_image_blocks`): el modelo tiene prohibido emitir URLs de fotografías y solo puede indicar el identificador del vehículo en un bloque de imagen; es el *backend* quien, en un paso posterior, recupera de la base de datos la URL real de la foto y la inserta, y descarta el bloque si el vehículo no existe o no tiene imagen. Para que esa URL no forme parte del esquema que se ofrece al modelo, el campo correspondiente se marca con `SkipJsonSchema`. La segunda afecta a las gráficas: se alimentan exclusivamente de la salida de la herramienta *agregar*, de modo que sus barras reflejan siempre valores calculados por la base de datos. Las etiquetas, unidades y títulos son metadatos de presentación que nunca modifican las cifras subyacentes, una separación que se documenta tanto en el modelo del *backend* como en su espejo del *frontend*.

6.5 FRONTEND CON REACT

La interfaz se ha implementado como una aplicación de página única con *React* (versión 19) y *TypeScript*, construida con la herramienta *Vite*. El uso de *TypeScript* permite definir un tipo `Car` (en `lib/api.ts`) que es el espejo exacto del modelo *Version* del *backend*, de modo que el contrato de datos queda verificado también en el lado del cliente. La capa de comunicación con la API se concentra en ese mismo módulo, que expone funciones tipadas para cada *endpoint* (`sendChat`, `fetchCars`, `fetchCar`, `fetchFiltersMeta`). La aplicación se organiza en torno a un componente `Layout` con una barra lateral que conmuta entre las cuatro vistas funcionales (asistente, catálogo, historial y favoritos), gobernadas por el estado de `App.tsx`.

6.5.1 Interfaz conversacional

La vista del asistente (`ChatAssistant.tsx`) gestiona el estado de la conversación y la comunicación con el *endpoint* `/chat`. Al enviar un mensaje, reconstruye el historial completo en el formato que espera el *backend*, serializando cada respuesta previa del asistente de nuevo a su JSON de bloques, y atiende los detalles de la interacción: muestra un indicador de escritura mientras se procesa la consulta, desplaza la vista automáticamente al último mensaje y, si la comunicación falla, presenta el error como un mensaje del propio asistente en lugar de como un fallo abrupto. Junto al campo de entrada figura de forma permanente el aviso de que la IA puede cometer errores.

El problema propio de esta vista es la renderización de las respuestas estructuradas, resuelto en `MessageBubble.tsx`. El componente recorre la lista de bloques y delega cada uno, según su tipo, en un sub-componente especializado: el texto se interpreta como *Markdown* (con la biblioteca `react-markdown` y soporte de tablas mediante `remark-gfm`); las tablas se pintan con desplazamiento horizontal; las gráficas se dibujan con la biblioteca *Recharts*; y las imágenes se muestran como fichas con un botón para añadir el vehículo a favoritos. Este despacho por tipo permite que una misma respuesta combine, por ejemplo, un párrafo, una tabla y una gráfica.

El componente de gráficas incorpora varias salvaguardas de presentación: recorta de forma determinista a un máximo de barras, oculta la leyenda cuando hay una sola serie, rota las etiquetas del eje cuando hay muchas categorías y, en las gráficas de tipo radar, normaliza cada eje a una escala común y muestra el valor real en la información emergente. Nada de esto altera los datos de origen, solo su presentación. Los colores se leen de variables CSS, de manera que las gráficas siguen el tema activo de la interfaz. El aspecto final de esta vista, con respuestas que combinan distintos tipos de bloque, se mostró en las figuras 5.8, 5.9 y 5.10 del capítulo anterior.

6.5.2 Catálogo, filtros y comparador

La vista de catálogo (`Catalog.tsx`) implementa la exploración estructurada. Al montarse, recupera del *backend* los metadatos de filtros y con ellos puebla dinámicamente los desplegables de marca, combustible, carrocería, tracción y transmisión, de modo que solo se ofrecen opciones que existen en la base de datos. Los filtros de rango y el buscador de modelo se combinan con la paginación, y la vista distingue de forma explícita los estados de carga, error y búsqueda sin resultados, y muestra en cada caso el componente adecuado (una rejilla de marcadores de posición durante la carga, un mensaje con opción de reintento ante un error, etc.). La separación entre los filtros que el usuario edita y los efectivamente aplicados evita relanzar la consulta en cada pulsación: la búsqueda solo se dispara al confirmar. El aspecto de esta vista se mostró en la figura 5.11 del capítulo anterior.

6.5.3 Favoritos e historial de consultas

Las funcionalidades de personalización (favoritos e historial) se resuelven íntegramente en el lado del cliente y persisten en el `localStorage` del navegador, sin cuentas de usuario ni almacenamiento en el servidor, en línea con el alcance del proyecto. Toda esta lógica se centraliza en `lib/storage.ts`.

El reto está en mantener todas las vistas sincronizadas con los cambios, incluso entre varias pestañas abiertas. Se resuelve integrando la lectura del almacenamiento con el sistema de reactividad de React a través del hook `useSyncExternalStore`: cualquier modificación de los favoritos invalida una caché interna y notifica a los componentes suscritos, que se vuelven a renderizar de inmediato; y un *listener* del evento `storage` del navegador propaga además los cambios realizados en otras pestañas. El historial agrupa las conversaciones en sesiones con un título derivado de la primera pregunta del usuario. Tanto la lectura de favoritos como la de sesiones es defensiva: se validan los datos leídos y se descartan las entradas con forma inválida, de modo que un `localStorage` corrupto no rompe la aplicación.

6.5.4 Preferencias de accesibilidad

El panel de preferencias (`SettingsDrawer.tsx`) se implementa como un cajón lateral accesible: declarado como diálogo modal (`role="dialog", aria-modal`), atrapa el foco dentro del panel ciclando con la tecla de tabulación, se cierra con la tecla de escape y devuelve el foco al control de origen al cerrarse. Reúne tres ajustes (tema, tamaño de fuente e idioma) gestionados por un contexto de React (`PreferencesContext.tsx`) que los persiste en `localStorage` con una lectura defensiva campo a campo.

Cada preferencia se traduce en un cambio sobre el documento raíz mediante funciones puras (`applyPreferences.ts`), reutilizadas tanto por el contexto como por el *script* de arranque: el tema conmuta un atributo en la raíz del documento que cambia la paleta de colores; el tamaño de fuente escala un único valor en la raíz, lo que, al estar toda la tipografía expresada en unidades relativas, redimensiona la interfaz proporcionalmente; y el idioma ajusta el atributo `lang` del documento y el sistema de internacionalización (*i18next*), que traduce los textos propios de la aplicación sin tocar nunca los datos de los vehículos. Queda una consideración sobre el arranque: en `main.tsx`, las preferencias guardadas se aplican al documento *antes* de montar React, lo que evita el destello visual que se produciría si la aplicación cargara primero con el tema por defecto y cambiara después. El panel de preferencias se mostró en la figura 5.14 del capítulo anterior.

6.6 CONTENERIZACIÓN Y DESPLIEGUE CON DOCKER

Todo el sistema se empaqueta y se orquesta con *Docker* y *Docker Compose*. Cada uno de los cuatro servicios (base de datos, *backend*, *frontend* y *scraper*) se ejecuta en su propio contenedor, definido por su respectivo `Dockerfile`, y todos comparten una misma red privada virtual (`appnet`) en la que se descubren por su nombre lógico.

Los `Dockerfile` del *backend*, el *frontend* y el *scraper* emplean construcciones multietapa con destinos diferenciados de desarrollo (`dev`) y producción (`prod`). Así se optimiza cada caso: en desarrollo, el *backend* arranca con recarga automática y el *frontend* con el servidor de desarrollo de Vite; en producción, el *backend* se ejecuta con varios procesos de trabajo y el *frontend* se compila a ficheros estáticos que sirve un servidor *nginx* ligero. Las imágenes parten de variantes reducidas (*slim*, *alpine*) y el *backend* ejecuta sus procesos con un usuario sin privilegios.

La orquestación se reparte en tres ficheros de Compose que se combinan: uno base (`docker-compose.yml`) con la definición común de los servicios, los volúmenes y la red; uno de *override* (`docker-compose.override.yml`), que se aplica automáticamente en desarrollo, publica los puertos y monta el código fuente como volumen para la recarga en caliente; y uno de producción (`docker-compose.prod.yml`), que selecciona los destinos `prod` y configura la política de reinicio. Esta separación permite levantar el sistema en uno u otro modo sin duplicar la configuración común.

El fichero base recoge además dos decisiones de orquestación del diseño. La primera es la persistencia: se definen dos volúmenes nombrados, uno para los datos de PostgreSQL (`pgdata`) y otro compartido entre el *scraper* y el *backend* (`scraper_output`), a través del cual el primero entrega el archivo de datos al segundo. La segunda es el arranque ordenado mediante *healthchecks* y dependencias condicionales: el *backend* no se inicia hasta que la base de datos supera su comprobación de salud (`pg_isready`), y ni el *frontend* ni el *scraper* arrancan hasta que el *backend* responde correctamente a su *endpoint* `/health`. Con ello se evitan los errores transitorios de arranque sin recurrir a esperas arbitrarias. Toda la configuración sensible (credenciales de la base de datos, clave del proveedor de IA, frecuencia del *scrapping*, orígenes CORS) se inyecta como variables de entorno, de modo que los secretos quedan fuera de las imágenes y del código.

La diferencia entre los modos de desarrollo y producción se concreta en el comando con el que arranca cada servicio, determinado por el destino multietapa seleccionado. La tabla 6.2 resume ambos casos.

Servicio	Desarrollo (dev)	Producción (prod)
bd	Imagen <code>postgres:16-alpine</code> (sin etapa multietapa)	
backend	<code>uvicorn -reload</code> (recarga en caliente)	<code>uvicorn -workers 4</code>
frontend	<code>vite dev</code> (servidor de desarrollo)	<code>nginx</code> (sirve estáticos compilados)
scraper	<code>sleep infinity</code> (inerte; <i>crawl</i> manual)	<code>supercronic</code> (cron que ejecuta <code>run_scrape.sh</code>)

Tabla 6.2: Comando de arranque de cada servicio según el destino multietapa (`dev/prod`) de su Dockerfile.

7 PRUEBAS Y EVALUACIÓN

Una vez descritas la implementación y la contenerización del sistema, este capítulo expone cómo se ha comprobado que *autoAI* funciona como se pretendía. Verificar un sistema con un componente de inteligencia artificial plantea una dificultad particular: junto a la lógica determinista, que se puede contrastar con resultados exactos, conviven comportamientos que no lo son, como la traducción de lenguaje natural a filtros o la variación deliberada de las respuestas exploratorias, sobre los que no cabe una aserción de igualdad. La estrategia adoptada distingue ambos planos. Para lo determinista (normalización, repositorio, validación de respuestas, tolerancia a fallos) se emplea la prueba automatizada con resultados exactos; para lo que no lo es, comprobaciones de invariantes y de propiedades junto con una evaluación cualitativa. El capítulo describe primero esa estrategia, después las pruebas del *backend* y del pipeline de IA y las del *frontend*, luego la evaluación de la calidad de las respuestas del asistente y, por último, una discusión de lo aprendido y de las limitaciones del enfoque.

7.1 ESTRATEGIA DE PRUEBAS

La estrategia de pruebas se ha guiado por un principio: respaldar con pruebas automatizadas las funcionalidades críticas, las que, de fallar silenciosamente, corromperían los datos o degradarían la experiencia, con un coste de mantenimiento proporcionado al de un Trabajo de Fin de Grado desarrollado por una sola persona. En lugar de perseguir una cobertura exhaustiva de cada línea, se han priorizado los puntos del sistema donde se concentra la complejidad y el riesgo: la normalización de datos heterogéneos, la construcción de consultas a la base de datos, la validación de las respuestas del modelo de lenguaje, la tolerancia a registros malformados durante la ingesta y los componentes de interfaz con lógica propia (paginación, persistencia local, renderizado de bloques). Este enfoque responde al requisito RNF-18, que exige pruebas automatizadas tanto en el *backend* como en el *frontend* para las funcionalidades de normalización, repositorio de datos y componentes de interfaz.

7.1.1 Niveles y tipos de prueba

Las pruebas se reparten en varios niveles, que se corresponden con las capas del sistema descritas en el diseño:

Pruebas unitarias. Verifican una unidad de código aislada, sin dependencias externas (ni base de datos ni red). Es el nivel más numeroso y abarca los *parsers* de normalización, las funciones puras de validación de modelos y los *helpers* de formato de la interfaz. Por no tocar recursos externos, son rápidas y deterministas.

Pruebas de integración. Comprueban la interacción real entre componentes; en este proyecto, fundamentalmente entre el repositorio de vehículos y una base de datos *PostgreSQL* real. Estas pruebas ejecutan SQL auténtico sobre un esquema cargado desde *init.sql*, de modo que validan el código Python y, con él, el propio esquema y las consultas.

Pruebas de componente (*frontend*). Renderizan componentes de React en un DOM simulado y comprueban su comportamiento ante la interacción del usuario (pulsaciones, navegación entre páginas), aislando las llamadas a la API mediante *mocks*.

Pruebas de extremo a extremo del pipeline. Ejercitan el *endpoint /chat* completo a través de un cliente de pruebas, sustituyendo únicamente la frontera externa (el proveedor del modelo de lenguaje), de forma que se recorre todo el flujo interno del *backend* —bucle de herramientas, generación del documento de bloques, validación, reintento, *fallback* e hidratación de imágenes— sin depender de un servicio externo.

Una decisión común a todos los niveles es el aislamiento de las fronteras no deterministas o costosas: el proveedor del modelo de lenguaje se sustituye siempre por un doble de prueba (*mock*) que devuelve respuestas controladas, y la red nunca se invoca durante las pruebas. Las pruebas resultan así reproducibles, rápidas y gratuitas, y permiten provocar a voluntad escenarios difíciles de reproducir con un modelo real, como una respuesta mal formada que dispare el reintento. La base de datos es la única dependencia externa que sí se usa de forma real, por dos razones: es determinista y aporta una garantía que un *mock* no podría dar, la de que el SQL generado es válido contra el motor concreto.

7.1.2 Herramientas e infraestructura de pruebas

Las pruebas del *backend* se han escrito con **Pytest**, acompañado de **pytest-cov** para la medición de cobertura. La configuración (recogida en *pytest.ini*) fija el directorio de pruebas, activa por defecto el informe de cobertura sobre el paquete *src* y define un marcador *integration* para distinguir las pruebas que requieren una base de datos real. Un fichero de configuración global (*conftest.py*) fija una cadena de conexión ficticia *antes* de que se importe la configuración de la aplicación, de modo que los modelos de configuración —donde la conexión a la base de datos es un parámetro obligatorio— puedan construirse sin un fichero de entorno real y sin abrir el *pool* de conexiones; las pruebas que no tocan la base de datos se ejecutan así sin ninguna infraestructura.

Las pruebas del *frontend* se han escrito con **Vitest** y **Testing Library** sobre un DOM simulado con *jsdom*, configurado en *vite.config.ts* y con un fichero de preparación común (*src/test/setup.ts*). Testing Library impone un estilo de prueba centrado en lo que percibe el usuario: los elementos se localizan por su rol y su texto accesible, en lugar de por detalles internos de implementación. Esto hace las pruebas más robustas frente a refactorizaciones y, de paso, ejercita la accesibilidad de los componentes.

La tabla 7.1 resume las herramientas de prueba empleadas en cada componente, qué frontera aíslan y qué niveles de prueba cubren.

Componente	Framework y utilidades	Aísla	Niveles
<i>Backend</i>	Pytest 8.3.4, pytest-cov 6.0.0, TestClient de FastAPI, <i>psycpg</i>	<i>Mock</i> del LLM; PostgreSQL 16 real	Unidad, integración, E2E, <i>smoke</i>
<i>Frontend</i>	Vitest 3, Testing Library, <i>user-event</i> , <i>jsdom</i> 25	<i>Mock</i> de la API (<i>fetch</i>)	Unidad, componente

Tabla 7.1: Herramientas de prueba por componente, fronteras aisladas y niveles cubiertos.

7.1.3 Pruebas como parte de la integración continua

Toda la batería de pruebas se ejecuta automáticamente en cada *push* al repositorio mediante el flujo de integración continua descrito en la Sección 4.4.2. El trabajo de *backend* levanta para ello un *service container* con *PostgreSQL* 16 y le apunta la cadena de conexión, de modo que las pruebas de integración (que en la máquina del desarrollador se omiten si no hay una base de datos accesible) sí se ejecutan en el servidor de integración contra una base de datos real. El trabajo de *frontend* ejecuta las pruebas con Vitest tras el *linter* y la construcción. Las pruebas dejan así de depender de una ejecución manual: se disparan con cada cambio y bloquean la consolidación de regresiones.

7.2 PRUEBAS DEL BACKEND Y DEL PIPELINE DE IA

El *backend* concentra la mayor parte de las pruebas del sistema, por ser el componente que aloja la lógica de negocio crítica. Se reparten en ocho módulos de prueba que cubren, de forma independiente, la normalización, el repositorio, la validación del pipeline de IA, la variedad de las búsquedas exploratorias, la tolerancia a fallos de la ingesta y los mecanismos de seguridad. La tabla 7.2 los resume.

Módulo de prueba	Qué verifica	Tipo
test_normalization_parsers	<i>Parsers</i> de saneamiento y unificación de vocabularios	Unidad
test_cars_repository_integration	Repositorio contra PostgreSQL real (filtros, orden, agregación)	Integr.
test_chat_pipeline	Validación, reintento, <i>fallback</i> e hidratación del <i>envelope</i>	Unidad / E2E
test_chat_validacion	Validación de los modelos de la petición de /chat	Unidad / E2E
test_buscar_coches_variedad	Determinismo y sesgo de la búsqueda exploratoria	Propiedad
test_ingest_tolerante	Tolerancia a registros malformados en la ingesta	Unidad
test_require_api_key	Autenticación opcional por clave (tiempo constante)	Unidad
test_smoke	Disponibilidad del <i>endpoint</i> /health	Smoke

Tabla 7.2: Módulos de prueba del *backend* y su cometido.

7.2.1 Pruebas de la normalización

La normalización es, como se argumentó en el diseño, una de las partes más delicadas del sistema, y por ello es la que recibe el mayor número de pruebas unitarias. Estas pruebas ejercitan los *parsers* puros (sin base de datos ni red) sobre los formatos reales que produce el portal de origen, comprobando tanto el camino feliz como los casos degenerados:

- El *saneamiento de escalares* se prueba con entradas tomadas de la fuente: de «177 CV / 130 kW» se exige la cifra en caballos (177); de «4.165 mm» el valor en milímetros sin el separador de miles; de «7,2 1/100 km» el número con la coma decimal convertida a punto; y de rangos de fechas como «(03/2021 - 06/2023)» las dos fechas que delimitan la vigencia. Para cada *parser* se comprueba además que una entrada vacía o el marcador de dato ausente (un guion) producen un valor nulo en lugar de un error, de acuerdo con el principio de que un dato problemático nunca debe abortar la conversión.
- La *unificación de vocabularios* se prueba sobre el caso más difícil, el del combustible: se verifica que un eléctrico sin combustión declarada se clasifica como *electrico*, que el nivel de electrificación se infiere del nombre comercial (microhíbrido, híbrido autorrecargable o híbrido enchufable) y que la base fósil se combina con ese nivel para dar el valor canónico, por ejemplo *phev_gasolina*. La traducción de las carrocerías del portal al vocabulario canónico se comprueba de manera análoga.
- La *idempotencia* se prueba explícitamente: aplicar la normalización a un valor ya canónico debe devolver ese mismo valor sin alterarlo, tanto para el combustible como para la carrocería. Esta propiedad garantiza que el catálogo pueda reimportarse sin corromper los datos ya normalizados.

7.2.2 Pruebas de integración del repositorio

El repositorio de vehículos se prueba contra una base de datos *PostgreSQL* real en lugar de contra un doble de prueba, porque su responsabilidad es generar SQL correcto: un *mock* no detectaría una consulta sintácticamente válida en Python pero errónea contra el motor. Las pruebas son autocontenidas. Un *fixture* recrea la tabla `cars` desde el mismo `init.sql` que usa producción e inserta un conjunto de datos sintético y conocido (catorce versiones con combustibles, carrocerías, precios, potencias, plazas y estados de vigencia elegidos a propósito), de modo que se pueden afirmar conteos y órdenes exactos. Si no hay una base de datos accesible, por ejemplo en la máquina del desarrollador sin Docker, un *fixture* de sesión omite todo el módulo en lugar de fallar; en la integración continua, donde el *service container* de *PostgreSQL* está vivo, las pruebas se ejecutan.

Sobre ese conjunto controlado se verifican las garantías del repositorio que sostienen los requisitos del catálogo y del asistente: el conteo total sin filtros, la paginación con páginas disjuntas (RF-01), los filtros de igualdad simple, de pertenencia a un conjunto y de rango (RF-02), el filtro de versiones vigentes mediante la condición de fecha de fin nula, la ordenación determinista por distintos criterios (RF-06), la recuperación de una ficha por su identificador y el caso de identificador inexistente (RF-04), la comparación que preserva el orden de entrada solicitado (RF-05), las agregaciones (recuento y media agrupada por una dimensión, RF-12) y los valores de los filtros que expone la base de datos para la interfaz (RF-03). Al partir de un *dataset* con valores escogidos, las aserciones son exactas: por ejemplo, que la media de precio de los eléctricos es la esperada, o que el filtro de potencia mínima devuelve exactamente el subconjunto previsto.

7.2.3 Pruebas del pipeline de IA

El pipeline de inteligencia artificial es, junto con la normalización, el componente con un comportamiento más difícil de verificar, porque su corazón es una llamada a un modelo de lenguaje externo no determinista. La estrategia consiste en aislar esa frontera, sustituyendo el cliente del modelo por un doble de prueba que devuelve respuestas predeterminadas, y en probar después toda la maquinaria interna del *backend* que rodea a esa llamada, que sí es determinista y sí es responsabilidad del sistema.

Las pruebas se organizan en dos planos. En el plano de unidad se verifican por separado las cuatro piezas del tratamiento de la respuesta del modelo, que materializan los requisitos RF-11 y RF-15:

- La *validación del documento de bloques (envelope)*: un documento bien formado se acepta y se reconoce el tipo de cada bloque; un contenido vacío, un JSON inválido o un bloque con un tipo desconocido provocan el error esperado. Queda así comprobado que el contrato de salida del asistente se hace cumplir sin excepciones.
- El *reintento*: ante una primera respuesta inválida, el sistema realiza exactamente una llamada adicional al modelo indicándole el error, y si esa segunda respuesta es válida la acepta; se verifica además que en la conversación se inserta la instrucción de corrección.

- El *fallback*: si el reintento tampoco produce una respuesta válida, el sistema devuelve un bloque de texto de reserva con un mensaje controlado, en lugar de propagar un fallo a la interfaz.
- La *hidratación de imágenes*: un bloque de imagen que solo contiene el identificador del vehículo se completa con la URL real de la foto recuperada de la base de datos; si el vehículo no existe o no tiene foto, el bloque se descarta sin afectar al resto del documento. La prueba confirma la invariante de que el modelo nunca emite URLs.

En el plano de extremo a extremo, las mismas situaciones se ejercitan a través del *endpoint* /chat con un cliente de pruebas, recorriendo todo el flujo interno: el bucle de herramientas, la generación del documento, la validación, el reintento, el *fallback* y la hidratación. Un doble de prueba configurable permite encadenar varias respuestas del modelo y simular, por ejemplo, una primera respuesta válida, una secuencia inválida-inválida que fuerza el *fallback* o una respuesta con un bloque de imagen que dispara la hidratación. Por separado se valida la capa de entrada del *endpoint*: que se rechazan los mensajes con el rol de sistema (para impedir que un cliente inyecte instrucciones de sistema), que se respetan los límites de longitud de los mensajes y de número de turnos, y que una lista de mensajes vacía produce un error controlado. Estas comprobaciones cubren la robustez del contrato de la API frente a entradas maliciosas o mal formadas.

Estas pruebas se entienden mejor sobre el flujo del pipeline ya descrito en el capítulo de diseño: el bucle de herramientas se corresponde con la figura 5.5, la generación y validación del documento de bloques —con su rama de reintento y *fallback*— con la figura 5.6, y la hidratación de imágenes con la figura 5.7. Cada una de las pruebas anteriores ejercita uno de esos tramos, de modo que, en conjunto, ninguna rama del tratamiento de la respuesta del modelo queda sin verificar.

7.2.4 Pruebas de propiedad de la búsqueda exploratoria

La búsqueda exploratoria, diseñada (Sección 5.4) para que dos consultas cualitativas parecidas no devuelvan siempre los mismos vehículos, es un caso especialmente ilustrativo. Aquí no cabe una aserción de igualdad, porque el resultado es variable a propósito, así que se recurre a pruebas de propiedad, que comprueban invariantes estadísticas en lugar de valores exactos. Sustituyendo el repositorio por un *pool* fijo de vehículos, se verifica que:

- cuando la consulta lleva un criterio de orden objetivo (la rama determinista), el resultado es estable byte a byte entre llamadas idénticas y respeta el orden pedido, sin barajar;
- cuando la consulta es exploratoria (sin orden), se devuelve el número de resultados solicitado, sin repetidos y todos pertenecientes al *pool*, y se solicita a la base de datos un conjunto de candidatos mayor que el límite, respetando un tope máximo;
- a lo largo de muchas ejecuciones con los mismos filtros aparece más de una combinación de resultados (la variación ocurre de hecho) y, aun así, el muestreo está sesgado hacia los mejores candidatos: el primer vehículo del *pool* aparece bastante más a menudo que el último. Esta última comprobación se formula con un margen holgado para que la prueba no sea frágil pese a depender del azar.

Este módulo muestra cómo verificar de forma automatizada un comportamiento no determinista a propósito: en vez de comprobar qué devuelve, se comprueban las propiedades que su distribución debe cumplir.

7.2.5 Pruebas de la ingesta tolerante a fallos y de la seguridad

La tolerancia a fallos de la ingesta (RF-22) se prueba alimentando al proceso un lote que mezcla registros válidos con registros defectuosos (uno al que le falta un campo obligatorio y un elemento que ni siquiera es un objeto) y sustituyendo el guardado en la base de datos por un doble de prueba. Se verifica que los registros correctos se procesan pese a los errores y que el informe resultante refleja con exactitud cuántos se leyeron, cuántos se validaron y cuántos se guardaron, además de identificar cada fallo por su posición en el lote. Queda así confirmado que un dato individual inválido no aborta la importación del conjunto, la garantía de robustez que exigía el diseño.

En cuanto a la seguridad, se prueba el mecanismo opcional de autenticación por clave de API (RNF-11). El caso de mayor interés es una petición sin clave cuando la autenticación está activada: la comprobación debe devolver un error de no autorizado y no un fallo interno, lo que obliga a tratar con cuidado la comparación en tiempo constante que evita fugas de información por temporización. Se verifican los cuatro escenarios (mecanismo inerte cuando no hay clave configurada, clave correcta, clave incorrecta y ausencia de clave) y, en los dos últimos, el código de error esperado. La protección frente a inyección de SQL (RNF-09) y la restricción de rutas en la ingesta (RNF-10) quedan cubiertas de forma indirecta: la primera, por las pruebas de integración del repositorio, que ejercitan la construcción parametrizada de consultas y la validación por listas blancas de los nombres de campo y de orden; la segunda, por el diseño del propio *endpoint*, que rechaza por construcción las rutas fuera del directorio autorizado.

7.3 PRUEBAS DEL *frontend*

Las pruebas del *frontend* se centran en los componentes y módulos con lógica propia, no en la presentación, con el mismo criterio de probar lo que entraña riesgo. Se agrupan en cinco módulos que cubren el formato de datos, la persistencia local, las tarjetas de vehículo, la paginación del catálogo y el renderizado de las respuestas estructuradas del asistente.

7.3.1 Formato de datos y persistencia local

Los *helpers* de formato se prueban como funciones puras: que un valor ausente se muestra como un guion en lugar de un cero o un vacío engañoso (coherente con el tratamiento de nulos del modelo de datos), que el precio se formatea como moneda sin decimales y con separador de miles, que las claves de combustible se traducen a su etiqueta legible y que el cero se trata como un valor válido y no como un dato ausente. Estas pruebas, en apariencia menores, blindan la frontera donde un nulo del *backend* podría convertirse en un dato incorrecto en pantalla.

La persistencia local (favoritos e historial, RF-16 y RF-17) recibe una atención especial porque es donde se concentra la lógica defensiva del cliente. Se prueba que una lectura de

un almacenamiento corrupto (un JSON inválido, un valor que no es una lista o entradas con forma inesperada) nunca rompe la aplicación: el dato se descarta con seguridad y se devuelve un valor vacío. Se prueba también que el marcado y desmarcado de favoritos se persiste; que las sesiones de historial se ordenan de la más reciente a la más antigua, que su título se deriva del primer mensaje recortándolo a una longitud máxima, y que al eliminar la sesión activa se limpia también la referencia a la sesión actual. Como el módulo de almacenamiento cachea los favoritos en estado de módulo para integrarse con el sistema de reactividad de React, cada prueba reimporta el módulo en limpio para partir de un estado conocido.

7.3.2 Componentes de interfaz

A nivel de componente se prueban tres piezas representativas:

- La tarjeta de vehículo (*CarCard*): que muestra la marca, el modelo, el precio formateado y el combustible legible, y que el botón de favorito refleja su estado mediante el atributo de accesibilidad correspondiente y, al pulsarlo, marca el vehículo y lo persiste. La prueba ejercita a la vez la presentación, la interacción y la integración con la persistencia local.
- El catálogo (*Catalog*): se centra en la lógica de paginación, que es la parte con más casos límite. Aislando las llamadas a la API, se verifica que una página llena habilita el botón de avance y deshabilita el de retroceso en la primera posición, que una página parcial deshabilita el avance, que al avanzar se solicita el desplazamiento correcto y se actualiza el número de página y, el caso más sutil, que cuando el total es múltiplo exacto del tamaño de página y la página siguiente llega vacía, el catálogo retrocede en lugar de mostrar una página en blanco.
- La burbuja de mensaje (*MessageBubble*): es la que renderiza las respuestas estructuradas (RF-11), de modo que se prueba el despacho por tipo de bloque: que un mensaje de usuario se muestra como texto plano; que un bloque de texto se interpreta como *Markdown* (el énfasis se convierte en el elemento HTML correspondiente); que un bloque de imagen con URL renderiza la imagen y sin URL no renderiza nada; y que los bloques de gráfica se dibujan correctamente, con detalles como la ocultación de la leyenda redundante cuando hay una sola serie o la normalización de las escalas en las gráficas de radar para que una magnitud grande no aplaste a las demás. Como la biblioteca de gráficas mide su contenedor, algo imposible en un DOM simulado, la prueba inyecta un tamaño fijo para que la gráfica se renderice y se pueda inspeccionar.

7.3.3 Cobertura de código

Como medida complementaria, que no sustituye al criterio de probar lo importante pero sí ayuda a detectar zonas sin verificar, el flujo de integración continua mide la cobertura de código del *backend* con *pytest-cov* y publica el informe como artefacto. Estas cifras conviene leerlas en su contexto: la cobertura es alta donde reside la lógica crítica respaldada por pruebas y baja donde aportaría poco. En la ejecución de referencia, la cobertura global del *backend* se sitúa en torno al 75 %, con los valores más altos en los módulos centrales: el repositorio de vehículos (*cars_repository.py*) alcanza el 94 %, la normalización (*normalization.py*) y

el pipeline de `/chat` (`api/chat.py`) rondan el 84 %, y los modelos de datos y la configuración quedan al 100 %.

Esta cifra del 75 % corresponde a la ejecución en la integración continua, donde las pruebas de integración del repositorio sí se ejecutan contra el *service container* de *PostgreSQL*; en una ejecución local sin base de datos, al omitirse esos diecisiete casos, la cobertura del repositorio cae y la global desciende a alrededor del 64 %. Los módulos con cobertura más baja son aquellos cuya verificación no se ha automatizado a propósito: el utilitario de análisis de variabilidad (`analizador_variabilidad.py`), una herramienta auxiliar usada durante el desarrollo del *scraping* y ajena al funcionamiento del sistema en producción, y algunas rutas y manejadores de la capa de API cuya lógica es fina y se ejercita de forma indirecta. La cobertura no es, por tanto, un objetivo en sí misma, sino un indicador de que el esfuerzo de prueba se concentró donde debía.

Coverage report: 75%

Files
Functions
Classes

coverage.py v7.14.1, created at 2026-06-19 19:21 +0000

File	statements	missing	excluded	coverage
src/_init_.py	0	0	0	100%
src/api/_init_.py	0	0	0	100%
src/api/cars.py	25	10	0	60%
src/api/chat.py	117	19	0	84%
src/api/dependencies.py	24	8	0	67%
src/api/routes.py	25	12	0	52%
src/core/_init_.py	0	0	0	100%
src/core/config.py	25	0	0	100%
src/core/db.py	8	0	0	100%
src/main.py	22	4	0	82%
src/models/_init_.py	0	0	0	100%
src/models/chat.py	28	0	0	100%
src/models/normalized.py	30	0	0	100%
src/models/scraped.py	27	0	0	100%
src/services/_init_.py	0	0	0	100%
src/services/ai_service.py	79	26	0	67%
src/services/cars_repository.py	97	6	0	94%
src/services/normalization.py	130	21	0	84%
src/utils/_init_.py	0	0	0	100%
src/utils/analizador_variabilidad.py	68	68	0	0%
Total	705	174	0	75%

coverage.py v7.14.1, created at 2026-06-19 19:21 +0000

Figura 7.1: Informe de cobertura de código del *backend* generado en la integración continua.

7.4 EVALUACIÓN DE LA CALIDAD DE LOS RESULTADOS

Más allá de la corrección del código, que las pruebas automatizadas garantizan, la calidad de *autoAI* depende de un aspecto que no admite una aserción binaria: la calidad de las respuestas del asistente conversacional. Que el sistema no falle es necesario, pero no suficiente; la pregunta relevante es si las recomendaciones son pertinentes, si las cifras son correctas y si la interpretación de las peticiones vagas es razonable. Esta sección describe cómo se ha evaluado esa dimensión.

7.4.1 La invariante de veracidad como criterio central

El criterio de calidad más importante del sistema es la invariante de veracidad (RF-09): toda cifra que aparece en una respuesta debe proceder de una consulta real a la base de datos, no de la imaginación del modelo. Esta invariante no se comprueba solo *a posteriori*: está garantizada por construcción en el diseño y respaldada por pruebas. El modelo no genera SQL ni redacta cifras libremente, sino que obtiene todos los datos a través del catálogo cerrado de herramientas, y los elementos más propensos a la invención (las URLs de las imágenes y las barras de las gráficas) se blindan con la hidratación desde la base de datos y con la alimentación exclusiva desde la herramienta de agregación, ambas cubiertas por las pruebas del pipeline. La evaluación de calidad parte, por tanto, de una base sólida: el diseño hace que mentir con datos sea, en lo posible, estructuralmente imposible.

7.4.2 Evaluación cualitativa mediante consultas representativas

La pertinencia y la naturalidad de las respuestas se han evaluado de forma cualitativa y manual, mediante una batería de consultas representativas ejecutadas contra el sistema durante la fase de refinamiento del pipeline. La batería se construyó para cubrir las distintas categorías de petición que el asistente debe atender, recogidas en los requisitos funcionales:

Consultas objetivas. Peticiones con un criterio inequívoco («el coche más barato», «el SUV más potente»), donde se comprobó que el resultado es determinista y correcto frente a la base de datos.

Consultas exploratorias y vagas. Peticiones cualitativas sin un orden objetivo («un coche para la familia», «algo para ciudad», «que gaste poco»), donde se evaluó que el asistente las traduce a filtros razonables (RF-10) y que la variación de resultados funciona sin degradar la calidad.

Consultas de comparación y de visualización. Peticiones que requieren comparar varios vehículos o agregar datos («precio medio por marca de los SUV»), donde se verificó que la respuesta combina los bloques adecuados (tabla, gráfica) y que las cifras coinciden con las de la base de datos (RF-12).

Consultas sobre atributos no soportados. Peticiones que mencionan características ausentes del modelo de datos (techo solar, color, etiqueta medioambiental), donde se comprobó que el asistente advierte de la limitación, propone la dimensión más cercana y ejecuta igualmente la búsqueda con el resto de criterios (RF-13), en lugar de rendirse.

Consultas multiturno y bilingües. Conversaciones de varios turnos, para comprobar el mantenimiento del contexto (RF-08), y peticiones en ambos idiomas, para verificar que solo se traduce la prosa y nunca los datos del vehículo (RF-14).

El propio capítulo de diseño ilustra el resultado de esta evaluación con capturas de la interfaz conversacional respondiendo con gráficas, tablas e imágenes (figuras 5.8 a 5.10). La evaluación fue iterativa: muchos de los ajustes del *prompt* de sistema y del comportamiento del pipeline descritos en el capítulo de implementación (el tratamiento de los híbridos enchufables, la agrupación de carrocerías, la variación de respuestas) surgieron de observar respuestas insatisfactorias en estas consultas y corregirlas, en un ciclo de prueba manual y refinamiento propio del desarrollo de sistemas con modelos de lenguaje.

La tabla 7.3 recoge una muestra de esa batería: para cada consulta representativa, su categoría, las herramientas que el asistente invocó, el tipo de bloques de la respuesta y la valoración cualitativa obtenida.

Consulta	Categoría	Tools	Bloques	Valoración
«El coche más barato»	Objetiva	buscar_coches	Texto, imagen	Correcta: devuelve el de menor precio de la BD.
«El SUV más potente»	Objetiva	buscar_coches	Texto, imagen	Correcta y determinista.
«Un coche para la familia»	Exploratoria	buscar_coches	Texto, imágenes	Pertinente: infiere plazas y maletero amplios; varía entre llamadas.
«Algo que gaste poco»	Exploratoria	buscar_coches	Texto, imágenes	Pertinente: filtra por consumo bajo.
«Compara el Golf y el León»	Comparación	buscar_coches, comparar	Texto, tabla	Correcta: tabla enfrentada con cifras de la BD.
«Precio medio por marca de los SUV»	Visualización	agregar	Texto, gráfica	Correcta: barras alimentadas por la agregación.
«Un coche con techo solar»	Atributo no soportado	buscar_coches	Texto, imágenes	Adecuada: advierte de la limitación y busca con el resto de criterios (RF-13).
«Dame un SUV» → «y más barato»	Multiturno	buscar_coches	Texto, imágenes	Correcta: mantiene el contexto entre turnos (RF-08).
«Recommend me a city car» (en inglés)	Bilingüe	buscar_coches	Texto, imágenes	Correcta: prosa en inglés, datos del vehículo intactos (RF-14).

Tabla 7.3: Muestra de la evaluación cualitativa del asistente sobre consultas representativas.

7.4.3 Cobertura de los requisitos por las pruebas

Las pruebas se corresponden con los requisitos definidos en el Capítulo 3. La tabla 7.4 traza esa correspondencia: para cada bloque de requisitos, indica el mecanismo de verificación empleado. Predominan las pruebas automatizadas en lo determinista y la evaluación cualitativa, apoyada en garantías de diseño, en lo que no lo es.

Requisitos	Mecanismo	Pruebas / evidencia
RF-01 a RF-06 (catálogo)	Integración (BD real)	<code>test_cars_repository_integration</code>
RF-07, RF-08, RF-15 (pipeline)	Unidad + E2E con <i>mock</i> del LLM	<code>test_chat_pipeline</code> , <code>test_chat_validacion</code>
RF-09 (veracidad)	Garantía de diseño + pruebas de hidratación	Tool-calling, hidratación de imágenes
RF-10, RF-13, RF-14 (interpretación)	Evaluación cualitativa manual	Batería de consultas representativas
RF-11 (bloques)	Componente (<i>frontend</i>)	<code>MessageBubble.test</code>
RF-12 (agregación/gráficas)	Integración + componente	<code>test_cars_repository...</code> , <code>MessageBubble.test</code>
RF-16, RF-17 (datos locales)	Unidad (<i>frontend</i>)	<code>storage.test</code>
RF-19 a RF-21 (normaliz.)	Unidad + integración	<code>test_normalization_parsers</code>
RF-22 (ingesta tolerante)	Unidad	<code>test_ingest_tolerante</code>
RNF-09 a RNF-11 (seguridad)	Unidad + integración	<code>test_require_api_key</code> , <code>repositorio</code>

Tabla 7.4: Trazabilidad entre requisitos y mecanismos de verificación.

7.5 DISCUSIÓN DE LOS RESULTADOS

El conjunto de pruebas reúne en total alrededor de ciento veinte casos (unos ochenta en el *backend* y unos cuarenta en el *frontend*), que se ejecutan automáticamente en cada cambio a través de la integración continua. Más relevante que el número es dónde se ha puesto el esfuerzo: las pruebas se han concentrado donde el riesgo y la complejidad son mayores (normalización, repositorio, pipeline de IA y persistencia local) y se han tratado de forma somera, o resuelto por diseño, las partes triviales o de pura presentación. La priorización es deliberada y proporcionada al contexto del proyecto.

De la experiencia de probar el sistema se extraen tres lecciones. Aislar las fronteras no deterministas (el modelo de lenguaje, la red) es lo que hace verificable un sistema con un componente de inteligencia artificial: al sustituir el modelo por un doble de prueba, todo el andamiaje que lo rodea, donde residen la mayoría de los fallos posibles, se vuelve determinista

y comprobable con aserciones exactas. Para los comportamientos variables a propósito, las pruebas de propiedad, que verifican invariantes estadísticas en lugar de valores concretos, resultan adecuadas, como ilustra la prueba del sesgo de la búsqueda exploratoria. Y algunas garantías se aseguran mejor por construcción que por prueba: la invariante de veracidad del asistente se sostiene menos por los casos de prueba que por un diseño que hace estructuralmente difícil violarla.

El enfoque tiene también limitaciones. La evaluación de la calidad de las respuestas del asistente es manual y cualitativa, no automatizada ni cuantificada con métricas formales; un trabajo más ambicioso podría incorporar una evaluación sistemática con un conjunto de consultas etiquetadas y, en su caso, un modelo evaluador. No se han realizado pruebas de carga ni de rendimiento bajo concurrencia, por quedar fuera del alcance de un prototipo de Trabajo de Fin de Grado, aunque el diseño contempla los índices y los límites necesarios para sostener un volumen razonable. Tampoco se han automatizado pruebas de extremo a extremo de la interfaz en un navegador real (con herramientas como *Playwright*), que ejercitarían el sistema completo desde el clic del usuario; las pruebas de componente cubren la lógica de la interfaz, pero no la integración visual de toda la aplicación. Estas limitaciones se retoman, como líneas de mejora, en el capítulo de conclusiones.

Con todo, el sistema de pruebas responde a su propósito: respalda las funcionalidades críticas con verificaciones automáticas y reproducibles, las asocia a los requisitos y constituye la red de seguridad que permitió desarrollar *autoAI* de forma incremental y refactorizar con confianza.

8 CONCLUSIONES Y TRABAJO FUTURO

8.1 CUMPLIMIENTO DE LOS OBJETIVOS

8.2 CONCLUSIONES

8.3 LÍNEAS DE TRABAJO FUTURO

BIBLIOGRAFÍA

A CONTENIDO DE UNA ENTRADA EN .BIB

Nombre campo	Descripción
address	Usualmente la dirección de la editorial
author	Nombre(s) del (de los) autor(es)
title	Título del libro
chapter	El número de un capítulo (o sección, etc)
edition	La edición de un libro, por ejemplo,segunda
editor	Nombre(s) del (de los) editor(es)
howpublished	Forma en que fue publicada la obra
institution	Institución responsable de un informe técnico
journal	Nombre del periódico o revista
key	Empleado para la alfabetización, referencias cruzadas y para crear una clave cuando la información del autor no está disponible. No debe confundirse con la etiqueta usada en el cite y que debe colocarse al inicio de la entrada
month	El mes de publicación o, para un trabajo inédito, en el que fue escrito
note	Cualquier información adicional que pueda ayudar al lector
number	El número del periódico, la revista, el informe técnico o del trabajo en una serie
organization	La organización responsable de una conferencia o que publica un manual
pages	Números de páginas
publisher	El nombre de la editorial. No debe confundirse con el editor
school	Nombre de la escuela donde fue escrita una tesis
series	El nombre de una serie o conjunto de libros
title	El título del trabajo
type	El tipo de un informe técnico
volume	El volumen de un periódico o una revista, o de algún libro que conste de volúmenes
year	El año de publicación. Para un trabajo inédito, el año en que fue escrito. Generalmente debe consistir de cuatro dígitos, por ejemplo 200

Tabla A.1: Campos de una entrada en archivo de bibliografía .bib

B TIPOS DE REFERENCIAS

article	Un artículo de un periódico o revista
book	Un libro con una editorial que se indica en forma explícita. Los campos requeridos en este caso son author (autor), editor, title (título), publisher (editorial) y year (año)
booklet	Una obra que está impresa y encuadernada, pero sin una editorial o institución patrocinadora
conference	Lo mismo que inproceedings, incluido para compatibilidad con el lenguaje de marcación Scribe
inbook	Una parte de un libro, que puede ser un capítulo (o sección) o un rango de páginas
incollection	Una parte de un libro que tiene su propio título
inproceedings	Un artículo en las actas de sesiones (proceedings) de una conferencia
manual	Documentación técnica
mastersthesis	Una tesis de maestría (Master thesis) o proyecto fin de carrera
misc	Para uso cuando los demás tipos no corresponden
phdthesis	Una tesis de doctorado (Ph D thesis)
proceedings	Las actas de sesiones de una conferencia
techreport	Un reporte publicado por una escuela u otra institución, usualmente numerado dentro de una serie
unpublished	Un documento que tiene un autor y título, pero que no fue formalmente publicado

Tabla B.1: Tipos posibles de elementos en bibliografía con natbib

ARTEFACTOS SOFTWARE

En este archivo se incluirá tanto el resumen en castellano como su traducción al inglés. Los dos párrafos estarán ligeramente separados.

El propósito del trabajo en una o dos frases. El diseño y metodología utilizada, los resultados más significativos del trabajo realizado y un breve resumen de las conclusiones. Debe ser conciso y presentar los resultados obtenidos tras la ejecución del TFG.

Put here the english translation Debe ser conciso y presentar los resultados obtenidos tras la ejecución del TFG. Los dos párrafos estarán ligeramente separados.

